队伍编号	MCB2601605
赛道	A 赛道

基于收缩集成与条件 Copula 的啤酒感官预测及风格生成

摘 要

啤酒风味同时受到化合物共变、风格差异和感官量表的影响。本文基于 250 款啤酒的 231 项测量与 50 维标准化感官数据，研究缺失条件下的感官评分、关键化合物识别及风格条件生成与迁移。

针对问题一，在化学测量存在缺失、感官目标为标准化连续值的条件下，建立**感官组收缩非负最小二乘（NNLS）堆叠模型**，利用极端随机树、直方图梯度提升、岭回归和偏最小二乘提取互补预测，将感官家族权重向共享权重收缩，以控制逐目标独立拟合的不稳定性。缺失填补和特征变换均在训练折内完成，通过**训练经验分布单调映射**形成相对强度，再就近展示为 0-6 七档相对评分。采用**主动集 NNLS** 求解，并以 **10 次重复 3 折外层验证**评价，重复宏平均**决定系数为 0.2564、平均绝对误差为 0.3361**。误差按标准化尺度计算，七档输出不等同于原始品评分恢复。

针对问题二，在化合物相关数据与实测特征均已给定、关键因素限定为严格单体的条件下，先以**化合物共表达网络**描述候选之间的共变，再建立**四路稳健秩融合模型**，综合总关联、分层自助、风格校正与留一风格名次，按支持数及倒数秩确定前三项。以化学身份、浓度与阈值的可比性及气味活性值核验合理性，感官模型条件归因仅作独立解释。经重采样与逐风格删除检验，在 218 个严格单体中给出 **50 维各 3 项、共 150 条关系**；bootstrap 入选频率中位数为 **0.395**，其中 **40 条不低于 0.60**，乙醇-酒精感、乳酸-酸度等 **8 条关系具有 A 级化学证据**。

针对问题三，在风格标签已知、目标风格样本较少且迁移须满足缺陷不恶化约束的条件下，建立**风格条件收缩 Copula 模型**，以经验边缘保留字段分布，用收缩相关矩阵描述化合物依赖，并调用感官预测器形成画像。对跨风格迁移建立**最小改动约束路径模型**，在源谱与目标端点之间搜索，将已知缺陷物质浓度不升纳入约束，陈旧酒花味用感官代理检验，未知项不判为通过。经**分位反变换抽样与有限候选枚举**，为 Tripel、Kriek、Lambic **各生成 300 条谱及画像**，训练折稳健尺度的边缘一阶 Wasserstein 距离为 **0.4762**。33 组迁移任务形成 **2 条同源待实测统计候选**，向 Tripel 和 Kriek 的目标距离分别下降 **38.36%和 37.36%**，但对代理裕量敏感。

折外验证、重采样、置换及裕量检验分别考察预测误差、关联稳定性与生成迁移边界。模型给出可核对的相对评分、关键关联和统计画像，迁移候选仍需独立测量与品评验证。

关键词：啤酒感官预测；非负堆叠；稳健秩融合；条件 Copula；约束迁移

目录

一、 问题重述 1
1.1 问题研究背景 1
1.2 问题回顾 1
二、 问题分析 1
2.1 问题一分析 1
2.2 问题二分析 1
2.3 问题三分析 1
三、 模型假设 2
四、 符号说明 3
五、 问题一的模型建立与求解 3
5.1 数据口径与特征 3
5.2 模型建立 4
5.3 模型求解 6
5.4 模型检验与结果讨论 9
六、 问题二的模型建立与求解 11
6.1 模型建立 11
6.2 模型求解 14
6.3 模型检验与结果讨论 18
七、 问题三的模型建立与求解 19
7.1 模型建立 19
7.2 模型求解 22
7.3 模型检验与结果讨论 25
八、 模型评价、改进与推广 27
8.1 模型的优点 27
8.2 模型的缺点 27
8.3 模型的改进与推广 27
参考文献 28
附录 A 支撑材料文件列表 29
附录 B 补充图片 30
附录 C 问题一模型核心程序 34
附录 D 问题二模型核心程序 38
附录 E 问题三模型核心程序 49

一、 问题重述

1.1 问题研究背景

啤酒的感官特征由多种化合物及其相互作用共同形成。同一化合物可能对应多个描述维度，风格差异又会改变化学组成与感官响应的分布。仅依据单变量相关筛选成分，容易混淆共线替代与风格效应；仅依据预测均值设计新谱，也无法保证生成样本保留真实化学依赖。因而，感官预测、关键成分识别和风格生成需要共用可核对的数据与预测接口，同时采用各自适当的验证证据。

1.2 问题回顾

问题一要求依据附件 1，给出附件 3 所定义 Aroma、Flavour 等维度的 0-6 分级评分。问题二要求利用附件 2 构建共表达图，结合附件 1，为每个维度给出 Top-3 化合物及稳定性和化学论证。问题三要求以 tasting_category_fine 为条件，生成 Tripel、Kriek、Lambic 等风格的化合物谱与感官画像，并在跨风格迁移时检查 DMS、双乙酰、4VG、反-2-壬烯醛及陈旧酒花味等缺陷。下文依次给出评分接口、50 维完整前三项结果及有约束的统计候选。

二、 问题分析

2.1 问题一分析

本问属于小样本、多输出连续回归。输入维数接近样本量，化学字段共线且各感官家族可预测性不同，单一高自由度模型易产生不稳定预测。本文以非线性树模型为基线，联合梯度提升、线性收缩和低秩多输出模型，通过感官组非负最小二乘（NNLS）堆叠与共享权重收缩控制方差。训练、变换和模型选择嵌入外层验证，并将连续误差、排序诊断及逐目标可靠性分开评价。附件感官值已标准化，原始评分不可唯一反演，故 0-6 只表示训练经验分布下的相对强度。

2.2 问题二分析

关键成分识别同时受共线替代、风格混杂和方法偏好影响。本文先界定严格单体候选，再以边际关联、稀疏回归和条件置换构造统计证据，将总关联、分层自助、风格校正及留一风格结果融合为稳健名次。第一问预测器的模块与字段归因用于检视模型依赖，化学身份和气味活性值用于核验合理性，二者均不回写最终排序。负对照、直接锚点和折内字段遮蔽分别检查噪声入选、已知关系恢复及预测敏感性，避免把统计前三项解释为因果或充分性结论。

2.3 问题三分析

风格生成需要同时保持字段边缘、联合依赖和样本离散程度，而目标风格样本稀少，不适合无约束估计高维协方差。本文以经验边缘与收缩正态 Copula 建立条件分布，以留出分布距离、几何覆盖及结构消融检验生成质量；随后调用第一问接口形成画像，并在可核验直接浓度与感官代理约束下搜索最小改动候选。未知浓度、检测地板和模型外推分别保留状态，通过固定候选裕量检验明确统计迁移的边界。

三问技术路线如图 1 所示。本文在统一数据口径下衔接感官预测、关键成分识别及风格生成迁移。第二问的正式 Top-3 由四路排序确定，NNLS 归因仅作独立解释；该排序不作为第三问优化输入。第三问调用同一预测器形成画像并检验代理约束，0-6 七档仅表示相对强度。

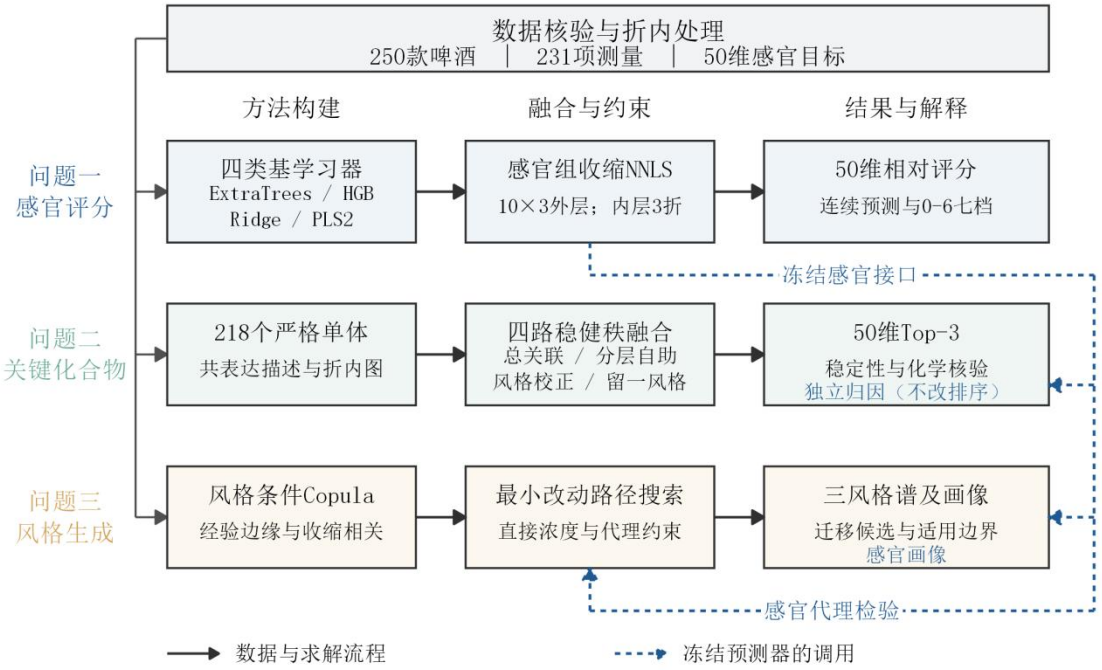


图 1 三问技术路线与感官预测接口

三、 模型假设

假设 1：按 beer 对齐后，每行代表一款可比较的啤酒，字段具有一致测量含义。附件主键唯一性与字段核对支持这一处理；若新增批次或重复样品，应改用批次分组验证并重新校准尺度。

假设 2：在当前已观测字段条件下，折内中位数填补与缺失指示可用于预测缺失项。该假设只服务于统计建模；若获得检测限或非随机缺失证据，应采用删失模型或分层处理，缺失浓度仍不得作为缺陷安全上限。

假设 3：附件标准化感官值可作为连续监督信号，训练折经验分布可用于相对等级展示。缺少原始评委中心与尺度时，不假设能恢复 0-6 原始均分；取得原始评分后，应重新进行量表校准。

假设 4：样本内化学-感官关系可在训练支持域附近用于预测与关联筛选。重复验证和风格扰动用于检验局部稳定性；若候选离开支持域或跨模型方向不一致，则降低解释强度，不推及未见风格或因果干预。

假设 5：风格经验边缘与收缩依赖足以构成生成的统计近似。若留出联合距离、相关诊断或离散程度恶化，应保留相应消融结果并限制画像用途；生成数量和随机种子不增加真实样本量。

假设 6：已知直接浓度约束与固定代理约束下的路径搜索可用于形成待实测候选。该假设不涵盖配方、发酵过程与物理安全；若要提出生产建议，需补充独立浓度测量、工艺可行性和感官品评。

四、 符号说明

表 1 汇总全文核心符号，训练统计量均仅由对应训练子样本估计。

表 1 全文主要符号及其含义

符号	定义	口径或范围
x_i；y_i	化学输入与标准化感官向量	231 字段；50 目标
N；J；p	啤酒数、感官目标数、化学字段数	250；50；231
T；V	当前外层训练集与验证集	互不重叠
$\widetilde{w}_g$；b_j；η	家族融合权重、目标截距、收缩系数	权重非负；收缩 0.5
$\hat{y}_{ij}$；$\hat{h}_{ij}$	连续预测与相对感官等级	等级 0-6，非原评分
H_{kj}；π_{kj}	四路融合分数与自助入选频率	频率 0-1
c_k；τ_k；OAV_k	校准浓度、阈值与气味活性值	前两者同单位
R_s；Q_{sk}	Copula 相关矩阵与经验分位函数	由训练风格估计
x_0；a；t	源谱、目标端点与迁移比例	比例 0-1
$d_s(x)$；$\Delta(x, x_0)$	目标距离与相对源谱的 RMS 改动	设计坐标，无量纲

五、 问题一的模型建立与求解

5.1 数据口径与特征

本文以附件 1 和附件 4 为共同数据入口：250 款啤酒具有 231 项化学及理化字段和 50 项标准化感官目标，分属 22 种细分风格。仅用 beer 连接，排除精度受损的 beer_id；最终预测器不使用风格标签，风格用于验证分层和误差报告。输入共有 73 个缺失单元，涉及 63 行、9 列；目标无缺失。表 2 列出清洗步骤，所有样本保留，未按全数据分位数剔除极端记录。

题面概述中的样本与维度数量同实际附件存在差异，本文以提供数据核对作答：附件 1/4 均为 250 款，附件 3 的 summary 实际含 50 个感官目标；经明确别名对齐后与附件 4 一一对应。附件 3 只在此核对定义，不作为额外观测或参与拟合。附件 4 缺少原品评尺度参数，评分输出采用式(6)的训练经验分布相对强度。

图 2 展示评分数据的结构：缺失集中在 9 字段，50 目标由五个家族组成，22 风格样本数明显不均衡。缺失数以单元计、风格样本数以实测啤酒计；附件 8 的 70 个尺度可核验字段不等于 70 字段逐值全部一致。

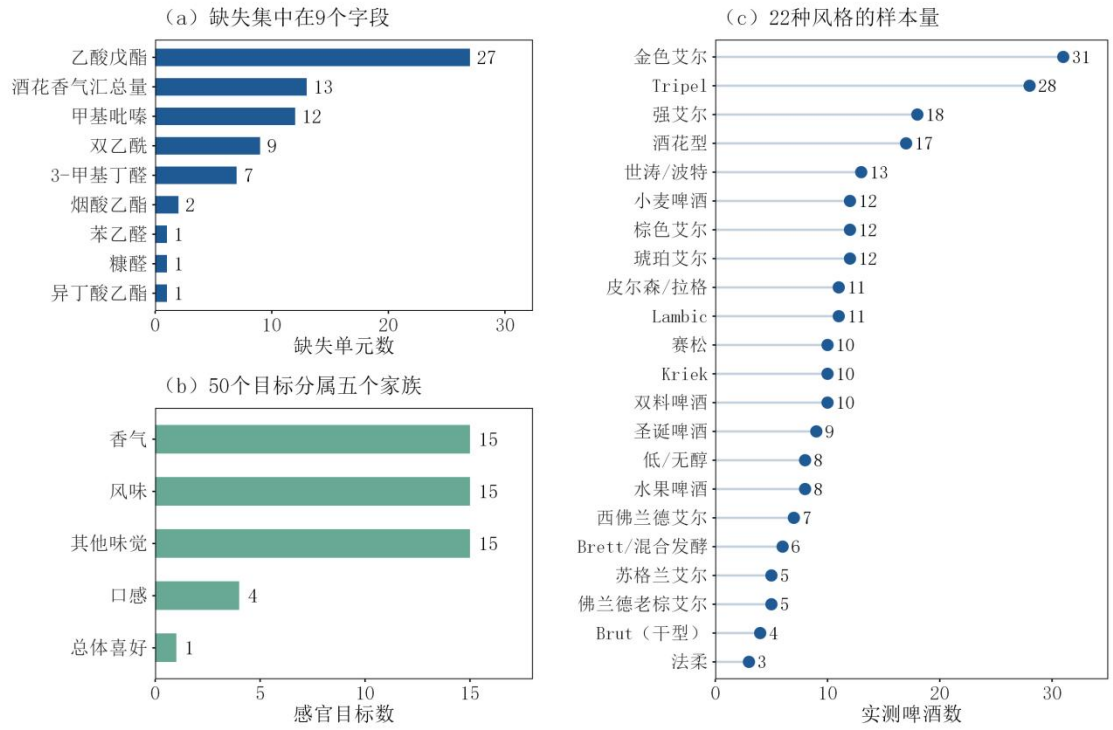


图 2 感官预测数据的缺失结构、目标构成与风格样本量

表 2 数据预处理步骤与维度变化

步骤	处理与范围	维度变化	理由
读取核对	附件 1 与附件 4	250×234；250×53	核对主键、目标与字段类型
样本对齐	按 beer 一一连接	250+250→250 行	避免标识精度导致错配
字段拆分	剔除三列元数据	X：250×231；Y：250×50	排除身份与风格标签入模
尺度保留	不重复取 log10、不全局截尾	250 行、231 输入不变	保留实际差异与极端观测
折内插补	训练中位数与缺失指示	231→231+k，0≤k≤9	73 单元、63 行、9 列缺失
分支变换	线性分支折内秩高斯变换	样本及目标数不变	减轻偏态、共线与量纲影响
相对映射	仅用训练折经验分布	50 维连续值＋50 维相对等级	不声称恢复原始评委均分

已识别的 log10 字段保持附件存储尺度，负值表示原浓度小于 1，不重复取对数。附件 8 可核验 37 个 log10 字段和 33 个原尺度字段，但尺度可识别不等于全部对应单元格一致；参考冲突单列保留，不静默替换输入。中位数填补、缺失指示及线性分支的秩高斯变换均由训练折拟合，0-6 映射亦仅依赖训练目标分布。后两问沿用这一数据契约，不再次定义主键、量表或预测接口。

5.2 模型建立

为同时利用非线性响应与多目标共性，以化学向量x_i预测标准化感官向量y_i，关系写为式(1)。误差项包含未测成分、品评差异及随机波动。基学习器选用极端随机树（ExtraTrees）[3]、直方图梯度提升（HGB）、秩高斯岭回归（Ridge）及偏最小二

乘多输出回归（PLS2）；它们分别刻画随机化交互、局部阈值、共线收缩与低秩共享，候选参数见表 3。

$$y_i = F(x_i) + \varepsilon_i \tag{1}$$

式中，i为啤酒样本索引；x_i为第i款啤酒的 231 维化学及理化字段向量，缺失和变换按训练折管道处理；y_i为对应的 50 维标准化感官向量，F为从化学组成到感官响应的多输出映射，ε_i为残差向量。记原字段维数p=231、感官目标数J=50、完整样本数N=250，后文k、j分别索引化学字段和感官目标。

表 3 基学习器与折内候选参数

学习器	候选范围	数据处理
ExtraTrees	300 棵；特征比例 0.33/0.60/1.00；叶节点 1/2/4/8；不限深度	中位数填补＋缺失指示
HGB	叶节点上限 7/15；最小叶样本 20；学习率 0.05；100 轮；L2=1	中位数填补＋缺失指示
Ridge	正则强度 0.1/1/10/100/1000	折内秩高斯变换与缩放
PLS2	潜变量数 2/5/10/15/20	折内秩高斯变换与多输出拟合

堆叠通过训练数据内部的折外基预测估计非负组合[4]。对训练目标和内层基预测分别中心化，再按训练目标标准差归一化，求解式(2)，避免高波动目标独占权重。五个家族预先定义为香气 15 维、风味 15 维、其他味觉 15 维、口感 4 维和总体喜好 1 维。

$$w_g^{*} = \arg\min_{w \ge 0} \sum_{j \in G_g} \sum_{i \in T} \left(\frac{y_{ij} - \bar{y}_{j,T} - \sum_{m=1}^{4} w_m \left(z_{ijm} - \bar{z}_{jm,T} \right)}{\sigma_{j,T}} \right)^2 \tag{2}$$

式中，T为当前外层训练样本集合，G_g为第g个感官家族的目标集合；m=1,…,4 索引四个基学习器，z_{ijm}为仅由训练集内部 3 折验证形成的第m个基预测。$\bar{y}_{j,T}$和$\bar{z}_{jm,T}$分别为当前训练目标及内层基预测均值，$\sigma_{j,T}$为目标总体标准差，取值不超过10^{-12}时按 1 处理。w_g^{*}为该家族未收缩的非负最小二乘权重；非负权重之和不受限为 1。

小家族可用信息较少，因此将家族解向全目标共享解收缩，如式(3)所示。收缩系数统一记为η并固定为 0.5；不使用外层成绩选取该值。非负约束作用于组合权重，不要求其和为 1。各目标的自由截距由训练均值恢复为式(4)，因此截距允许为负。

$$\tilde{w}_g = (1 - \eta) w_g^{*} + \eta w_{\mathrm{all}}^{*}, \eta = 0.5 \tag{3}$$

式中，$\tilde{w}_g$为第g个感官家族最终采用的权重，w_{all}^{*}为将全部 50 个目标共同中心化和标准化后求得的共享非负权重，η为预先固定的收缩系数，取 0.5。

命题 5-1　家族收缩的训练损失界。固定式(2)的训练预测及尺度，将其非负二次目标记为$\mathcal{L}_g$。式(3)的最终家族权重仍非负，且其家族训练损失不超过共享权重的家族损失。

证明。式(2)为仿射残差平方和，因而是权重的凸函数；家族最小化解的损失不大于任意可行共享解。式(3)是两个非负向量的凸组合，由凸性得$\mathcal{L}_g(\tilde{w}_g) \le (1 - \eta)\mathcal{L}_g(w_g^{*}) + \eta \mathcal{L}_g(w_{\mathrm{all}}^{*}) \le \mathcal{L}_g(w_{\mathrm{all}}^{*})$。该结论只控制固定训练目标的损失，不保证外层预测指标提高。□

$$b_j = \bar{y}_{j,T} - \sum_{m=1}^{4} \tilde{w}_{g(j),m} \bar{z}_{jm,T} \tag{4}$$

式中，b_j为第j个感官目标的自由截距，$g(j)$为该目标所属感官家族，$\tilde{w}_{g(j),m}$为对应家族权重的第m分量。截距由当前训练均值恢复，允许为负。

各基学习器在完整外层训练集上重新拟合后，验证样本按式(5)预测。表 4 给出 30 个外层折的平均权重；权重反映预测组合，不解释为化合物或机制的因果贡献。

$$\hat{y}_{ij} = b_j + \sum_{m=1}^{4} \widetilde{w}_{g(j),m} f_{mj}^{(T)}(x_i) \tag{5}$$

式中，$\hat{y}_{ij}$为第i款外层验证啤酒在第j个目标上的连续预测，$f_{mj}^{(T)}$为第m个基学习器在完整外层训练集T上拟合后的第j个输出分量。该分量记号避免将标量截距与整个多输出预测向量相加。

表 4 感官组收缩堆叠的平均非负权重

感官家族	ExtraTrees	HGB	Ridge	PLS2
香气	0.713	0.142	0.040	0.231
风味	0.784	0.146	0.033	0.202
其他味觉	0.648	0.250	0.100	0.178
口感	0.643	0.207	0.172	0.219
总体喜好	0.642	0.180	0.046	0.192

附件 4 的标准化聚合值缺少评委中心与尺度参数，不能唯一还原原始评分。为展示维度内相对强度，使用训练目标经验分布函数$\mathcal{F}_{j,T}$，按式(6)映射预测。先逐折映射，再平均重复等级；不能用验证观测或全数据分布拟合映射。该接口为后续画像提供相对等级，连续尺度仍承担训练和误差评价。

$$\hat{h}_{ij} = 6 \cdot \mathcal{F}_{j,T}\left(\hat{y}_{ij}\right), 0 \le \hat{h}_{ij} \le 6 \tag{6}$$

式中，$\mathcal{F}_{j,T}$为当前训练目标中不大于给定值的样本比例，$\hat{h}_{ij}$为映射后的相对强度，范围为 0-6。该值仅表示同一感官维度的训练经验分位，不是由标准化值唯一恢复的原始品评分。

为对应题面的七档分级展示，对已经固定的相对强度再定义$h_{ij}^{\mathrm{disc}} = \left\lfloor \hat{h}_{ij} + 0.5 \right\rfloor$，即就近取 0-6 整数、半档向上。该整数仅是相对分位的展示码，连续预测及此前所有评价数字保持原口径；不能把排序代理指标称为原始评分准确率。完整 250×50 相对强度与七档展示随评分结果表提供。

5.3 模型求解

本文用 scikit-learn 实现基学习器：ExtraTrees 采用随机切分建树，HGB 采用直方图梯度提升，Ridge 采用自动线性代数求解，PLS2 采用 NIPALS 迭代。堆叠调用 SciPy 主动集 NNLS，随后作 0.5 收缩并恢复自由截距；参数由内层 3 折选择，外层 10×3 划分用于评价。

外层按 tasting_category_fine 分层，采用 RepeatedStratifiedKFold，参数为 n_splits=3、n_repeats=10 和 random_state=20261023；样本最少的 Faro 有 3 款，每个验证折各 1 款。以 0 为起点编号重复和外折，内层采用 KFold，设置 n_splits=3、shuffle=True，种子为 $20261023 + 10r + f$，其中$r = 0, \ldots, 9$、$f = 0,1,2$。基学习器、候选参数与内折的位置均从 0 计数；随机种子在内层种子上加 100000 倍基学习器位置、1000 倍候选位置及内折位置，完整外层重拟合仅加 100000 倍基学习器位置。

秩高斯变换采用 QuantileTransformer(output_distribution="normal", subsample=None)，n_quantiles=min(100, 当前训练样本数)。PLS2 使用 scale=True、max_iter=1000、tol=10^{-6}。基学习器候选按 MAE 从小到大选择，同分时依次比较R^2、Spearman、代理 QWK 和相邻等级一致率，四项均由大到小；该内层选参顺序与表 5 中最终结构的综合比较分开。

采用固定 10 次重复、每次 3 折的风格分层外层划分，每款啤酒在每次重复中恰作为一次验证样本。外层训练内部完成参数选择、基预测构造、共享及家族 NNLS 估计、权重收缩和截距恢复。各目标先单独评价，再等权平均；宏平均决定系数（R^2）与平均绝对误差（MAE）分别由式(7)和式(8)定义。均方根误差（RMSE）同样先按目标计算，避免不同波动维度隐含加权。

$$R^2{}_{\text{macro}} = \frac{1}{J}\sum_{j=1}^{J}\left[1 - \frac{\sum_{i=1}^{N}\left(y_{ij} - \hat{y}_{ij}\right)^2}{\sum_{i=1}^{N}\left(y_{ij} - \overline{y}_{j,\text{eval}}\right)^2}\right] \tag{7}$$

式中，$R^2{}_{\text{macro}}$为先对每个目标计算再等权平均的决定系数；N=250 为当前一次重复中完整折外评价的啤酒数，J=50 为目标数，$\overline{y}_{j,\text{eval}}$为这$N$个真实目标值的均值。该评价均值与模型拟合时的训练均值$\overline{y}_{j,T}$分别计算。

$$\text{MAE}_{\text{macro}} = \frac{1}{NJ}\sum_{j=1}^{J}\sum_{i=1}^{N}\left|y_{ij} - \hat{y}_{ij}\right| \tag{8}$$

式中，$\text{MAE}_{\text{macro}}$为 50 个目标绝对误差的等权平均，分母为全部评价单元的数量，误差以附件标准化感官单位计。

Spearman 检验连续排序；七档分位代理二次加权 Kappa（QWK）及相邻等级一致率分别将观测和预测按各自样本分位数分档，不作为监督标签，仅供排序一致性诊断及内层选参的平局判据，不能称原始 0-6 评分准确率。表 5 和图 3 比较同一外层划分下的模型。感官组收缩NNLS的重复均值R^2、MAE、RMSE、Spearman分别为0.2564、0.3361、0.4714、0.4302，代理 QWK 为 0.4183，相邻等级一致率 55.42%。五项主比较指标均在 10 次重复中优于 ExtraTrees 基线。

表 5 相同外层划分下的五项预测指标

方法	R^2（均值±SD）	MAE（均值±SD）	Spearman	代理 QWK	相邻等级一致率
ExtraTrees 基线	0.2359±0.0058	0.3422±0.0011	0.4144	0.4026	54.76%
连续凸融合	0.2482±0.0065	0.3397±0.0013	0.4273	0.4150	55.23%
连续-有序融合	0.2480±0.0060	0.3356±0.0014	0.4314	0.4194	55.52%
逐目标收缩融合	0.2472±0.0059	0.3360±0.0012	0.4295	0.4172	55.25%
感官组收缩 NNLS	0.2564±0.0070	0.3361±0.0016	0.4302	0.4183	55.42%

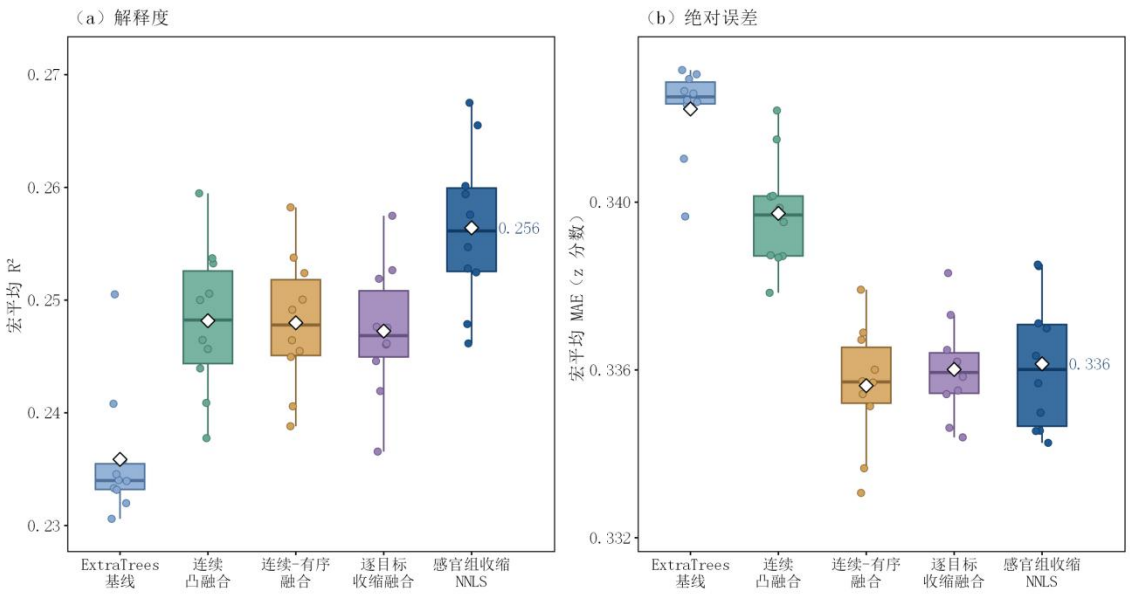


图 3 基线、融合与最终模型的重复折外性能比较（纵轴为局部范围）

连续-有序融合的 MAE 为 0.3356、Spearman 为 0.4314、代理 QWK 为 0.4194、相邻等级一致率 55.52%，均略优于最终模型；最终选择依据解释度、平均折外精度及稳健性综合权衡。先平均 10 次折外预测再评价时，最终模型 R^2/MAE/RMSE/Spearman 为 0.2742/0.3315/0.4655/0.4487，代理 QWK 和相邻等级一致率为 0.4368、56.42%；基线R^2/MAE 为 0.2502/0.3389。该口径降低划分波动，不能替代逐重复均值；逐目标收缩融合的平均折外 Spearman 仍以 0.4511 略高。

最终选择侧重连续感官信号的解释度，同时检查尾部目标和风格稳健性：表 5 中感官组收缩 NNLS 的R^2最高，而连续-有序融合的四项误差或排序指标略优，故不将最终方法表述为单项全面占优。这些差异来自同一组外层划分的方法比较，不是独立最终测试；表 6 只比较最终模型与基线，不能据此判定其相对连续-有序融合的四项微小差异显著。

图 4 展示全部目标，而图 5 采用预先固定的酸度、4VG 和陈旧酒花味检视不同信号强度。酸度等目标的观测-预测关系较集中，弱目标则向均值收缩；排序尚可并不必然对应较高解释度。图中拟合带是均值置信带，不是单个新样本预测区间。逐目标可靠性见附图 S3；固定目标残差与相对量表映射见附图 S1。

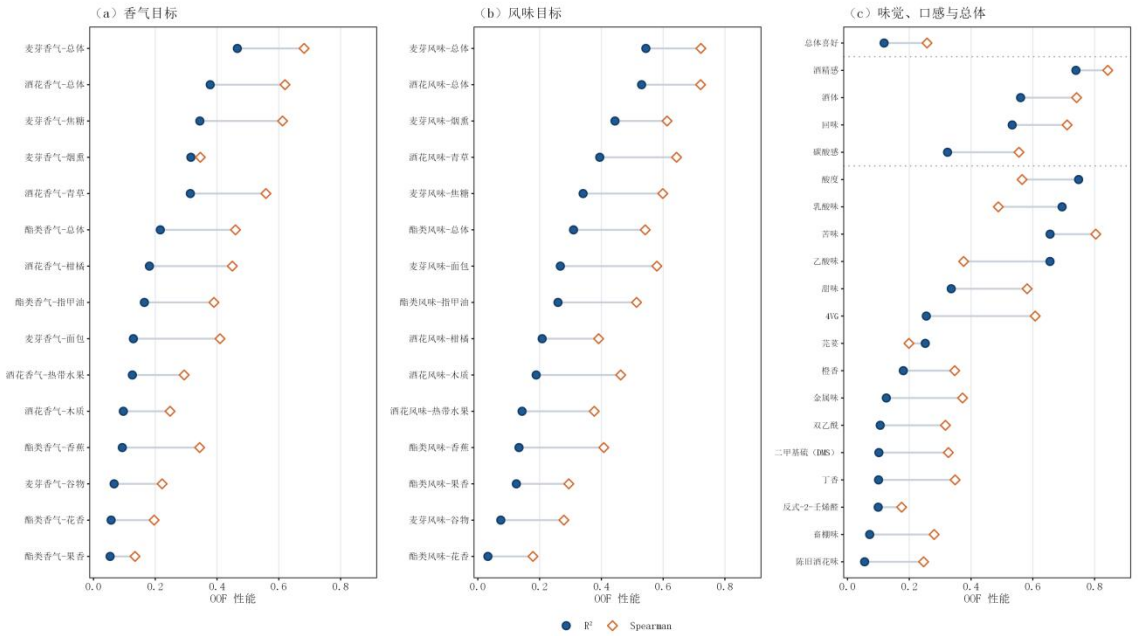


图 4 最终模型 50 个感官目标的平均折外R^2与 Spearman

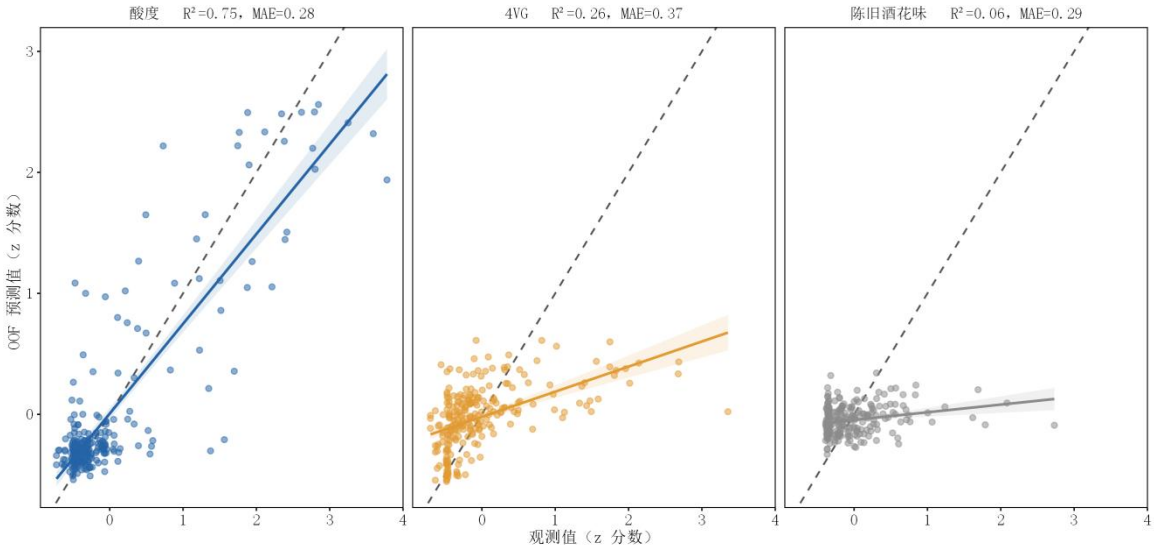


图 5 固定代表感官目标的平均折外观测-预测关系

5.4 模型检验与结果讨论

先检查数据和设定一致性：125000 个外层预测全部有限，beer、目标及折分身份与原数据一致；填补、变换、选择、堆叠和映射均限制在训练范围，附件 2、5、6 的全量相关矩阵不参与本问筛选。官方聚合标签的上游标准化因缺少原评委记录尚无法独立核验，故这里验证的是给定监督信号下的样本内泛化。

尾部目标在基线下预先固定，不能随最终成绩重选。最低 13 维平均R^2从 0.0411 升至 0.0639、MAE 从 0.3277 降至 0.3227；10 个预登记难目标平均ΔR^2为+0.0081、ΔMAE 为−0.0032。图 6 同时保留正负变化：22 风格中 20 个重复平均 MAE 下降，最差 Brut 由 0.5512 降至 0.5348；佛兰德老棕艾尔和 Tripel 分别增加 0.0019、0.0011。总体喜好及反-2-壬烯醛R^2分别下降 0.0144、0.0101，不能由宏平均改善推断所有目标受益。

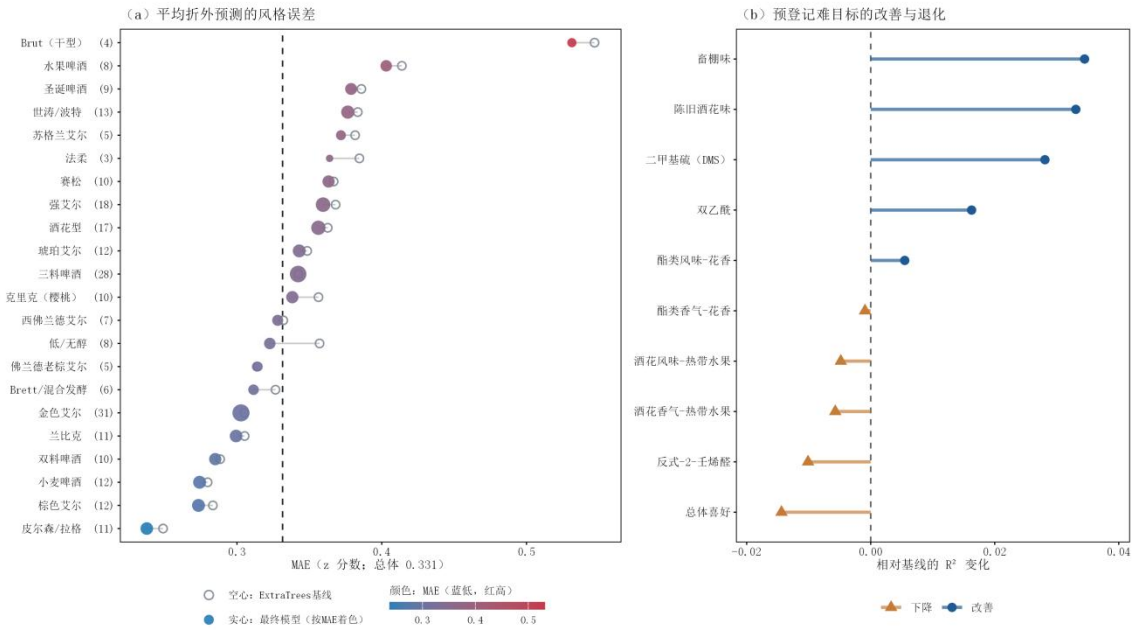


图 6 平均折外风格误差与预登记难目标的重复平均R^2变化

固定平均折外预测后，以啤酒为配对单位进行 20000 次自助抽样，表 6 中的 MAE 和归一化绝对秩误差差值区间均低于 0。该结果支持固定预测下的平均误差改善，不包含重新训练或选择模型的不确定性。表 7 进一步比较固定η=0.5 时的三种共享结构；组收缩的解释度和 MAE 较优，逐目标结构的相邻等级一致率略高。因此该表支持结构权衡，不是收缩系数邻域的灵敏度曲线。

表 6 最终模型相对基线的配对自助重采样结果

指标	基线	最终模型	差值	95%百分位区间
MAE	0.338879	0.331498	-0.007381	[-0.009467, -0.005331]
归一化绝对秩误差	0.237023	0.233291	-0.003732	[-0.006055, -0.001502]

表 7 权重共享结构消融

结构	R^2	MAE	Spearman	代理 QWK	相邻等级一致率
共享权重 NNLS	0.2551	0.3365	0.4298	0.4175	55.34%
感官组收缩 NNLS	0.2564	0.3361	0.4302	0.4183	55.42%
逐目标收缩 NNLS	0.2551	0.3364	0.4274	0.4157	55.43%

表 8 与图 7 检验校准及不确定性。重复宏R^2为 0.2564±0.0070、MAE 为 0.3361±0.0016，±表示重复样本标准差；平均五分位分箱校准误差（ECE）为 0.0516。重复 SD 区间在名义 80/90/95%下仅覆盖 18.0/23.2/27.5%，反映拟合波动远小于总预测误差。其他 9 次重复残差校准后为 80.0/90.0/95.0%，但重复使用同批 250 个真值，仅是样本内校准，不能当独立覆盖保证。

表 8 最终模型的补充检验结果

检验项	核验结果	单位/口径
折外完整性	10×250×50，全部有限	每款每重复恰好验证一次
重复宏R^2	0.2564±0.0070	10 次重复均值±样本 SD
重复宏 MAE	0.3361±0.0016	附件 4 标准化尺度
五分位箱校准	平均 ECE=0.0516	逐目标平均折外预测，5 箱
重复 SD 区间覆盖	18.0% / 23.2% / 27.5%	名义 80% / 90% / 95%
残差校准区间覆盖	80.0% / 90.0% / 95.0%	同批真值，样本内校准
逐目标R^2	0.033-0.748；中位 0.212	先平均 10 次折外预测
相对可靠性分层	较高 17 / 中等 16 / 较低 17	当前 50 目标秩三分位
留一风格压力测试	Spearman −0.04-0.41；MAE 0.26-0.54	22 风格；n=3-31

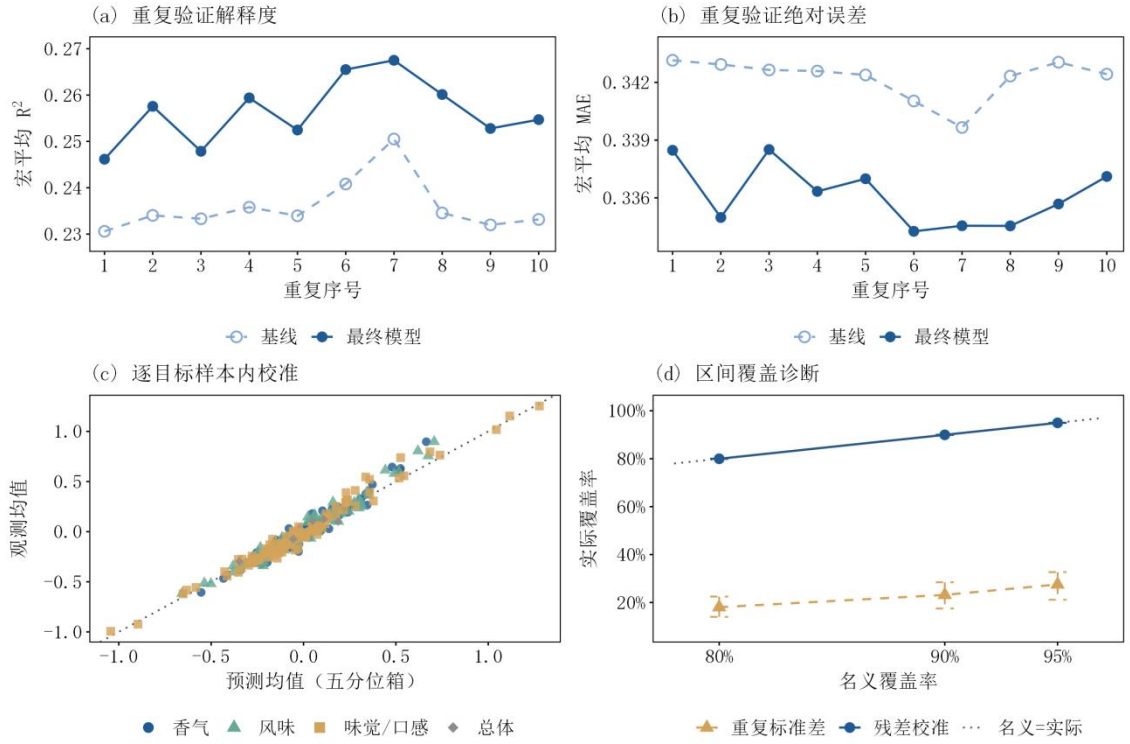


图 7 重复验证稳定性与预测校准

平均折外预测逐目标R^2范围 0.033-0.748、中位 0.212，17/16/17 为相对三分位，不是固定实用性标准。单次重复下，完整模型R^2/MAE 为 0.2462/0.3385，移除 4 个汇总字段为 0.2453/0.3382；全局一次 beer 目标行置换后为−0.0094/0.4176。留一风格 Spearman 为−0.04-0.41、MAE 为 0.26-0.54，小风格R^2受留出方差影响，仅描述前两项。新增重训未保存逐样本预测，数字只经源表、实现与运行记录核对；不能称已从预测独立复算，也不据此排除全部泄漏风险。结构消融与留一风格压力检验见附图 S2。

六、 问题二的模型建立与求解

6.1 模型建立

本问在第 5 章数据契约与预测接口上识别关联成分。先排除 13 个不满足严格单体资格的字段，保留 218 个严格单体候选，资格先于排序确定。表 9 明确数据与证

据边界；式(9)表示第 j 个感官目标的预测关系，最终名次由统计证据融合给出，不直接等同于某个预测器的重要性。

图 8 给出问题二的实际候选口径。231 个字段中先排除 13 个不满足严格单体资格的字段，再对 218 个候选的 23653 对相关作描述，成对有效样本为 211-250；最终 50 维各选 3 项得到 150 条关系，不能将其误作 150 个输入字段。附件 2 虽标称 Pearson，其数值与附件 1 成对完整 Spearman 核验一致，用于本文共表达描述图；监督检验的相关与模块仍在训练折重算。

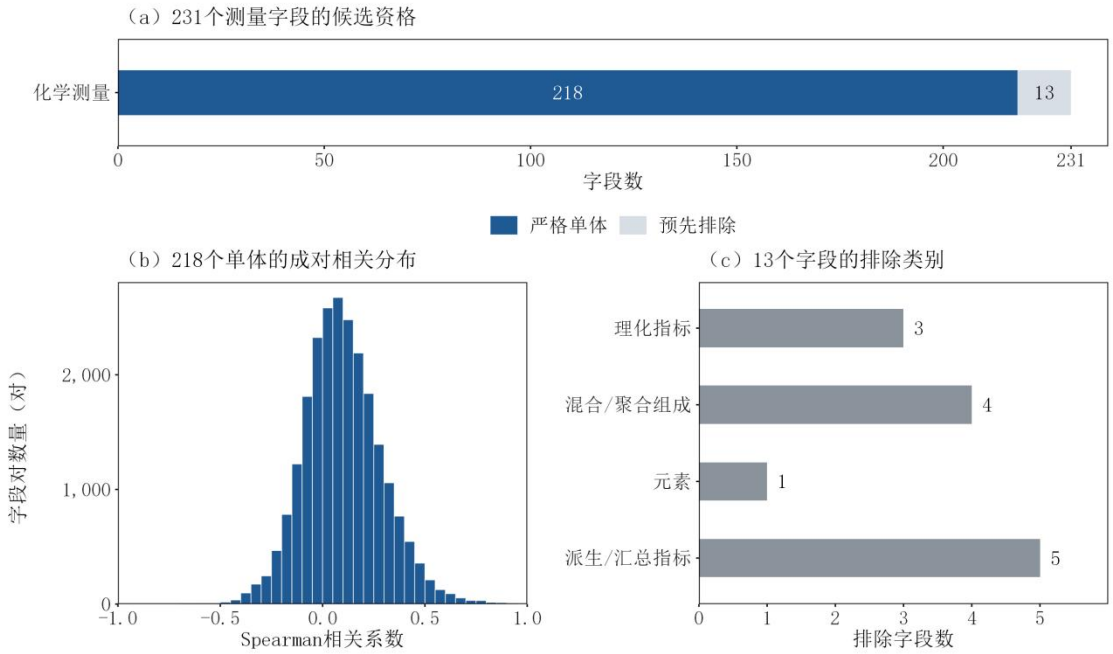


图 8 单体候选资格及其实际共线性分布

表 9 问题二数据契约与证据边界

项目	核验结果	处理方式	解释边界
样本与目标	250 款、50 维	固定 10×3 外层划分	逐目标折外评价
输入字段	231 项、73 个缺失	训练子样本内填补	不删除含缺失样本
严格候选	218 个单体	排除 13 个非严格单体字段	全因素仅作敏感性
共表达矩阵	231×231 Spearman	描述图与折内图分开	验证时不复用全量图
感官值	标准化连续值	按附件尺度建模	不声称恢复原始 0-6 分
化学证据	阈值、来源、异味、文献	排名固定后追加	不作因果解释

$$y_{ij} = F_j(x_i) + \varepsilon_{ij} \tag{9}$$

式中，F_j为前述多输出映射F的第j个目标分量，ε_{ij}为相应单目标残差。本式只是感官预测关系的分量写法，候选化合物的正式次序仍由四路统计证据决定。

第一类证据采用成对完整观测的 Spearman 边际关联，按式(10)计算秩协变；共表达网络用于描述共变与模块[5]。第二类采用弹性网（ElasticNet）与有符号图二次惩罚，式(11)同时包含 L1 稀疏、L2 收缩和图平滑项。图及全部预处理在训练折估计，条件依赖估计参照 graphical lasso 思想[6]，全量描述图不参与监督验证。

$$\rho_{kj}=\frac{\sum_{i\in\mathcal{A}_{kj}}\left(R_{ik}^{\mathrm{X}}-\overline{R}_{k}^{\mathrm{X}}\right)\left(R_{ij}^{\mathrm{Y}}-\overline{R}_{j}^{\mathrm{Y}}\right)}{\sqrt{\sum_{i\in\mathcal{A}_{kj}}\left(R_{ik}^{\mathrm{X}}-\overline{R}_{k}^{\mathrm{X}}\right)^{2}\sum_{i\in\mathcal{A}_{kj}}\left(R_{ij}^{\mathrm{Y}}-\overline{R}_{j}^{\mathrm{Y}}\right)^{2}}}\tag{10}$$

式中，$\mathcal{A}_{kj}$为当前训练集合T中字段k与目标j双方完整观测的样本下标集合，n_{kj}为该集合大小；ρ_{kj}为这组有效配对观测的 Spearman 秩相关，R_{ik}^{X}和R_{ij}^{Y}分别为该对子集中的化学字段秩与感官目标秩，$\overline{R}_{k}^{\mathrm{X}}$和$\overline{R}_{j}^{\mathrm{Y}}$为对应平均秩。并列观测采用平均秩，不以插补值增补相关样本。

$$\widehat{\beta}_{j}=\arg\min_{\beta}\left\{\frac{\left\|y_{j,T}^{\mathrm{c}}-X_{T}^{\mathrm{c}}\beta\right\|_{2}^{2}}{2n_{T}}+\lambda_{1}\|\beta\|_{1}+\lambda_{2}\|\beta\|_{2}^{2}+\lambda_{G}\beta^{\mathrm{T}}L\beta\right\}\tag{11}$$

式中，X_{T}^{c}和$y_{j,T}^{\mathrm{c}}$分别是按训练列均值中心化后的设计矩阵与目标列，β为待估系数向量，$\widehat{\beta}_{j}$为其解。矩阵有n_{T}行、q列，$n_{T}=|T|$，q包括 231 个测量字段及本折缺失指示，不等于 218 个单体资格。L为拓展至这些列的有符号归一化图拉普拉斯，缺失指示不连接化学边；λ_{1}、λ_{2}、λ_{G}依次控制 L1、L2 及图惩罚，截距在拟合后恢复。

非线性证据采用模块联合及模块内条件置换，处理共线字段间的替代[7]。归因固定第 5 章同折预测器的基模型、五个感官家族权重和截距，不重新选参或估计 NNLS。先整组置换折内模块，再逐字段条件置换且保留缺失掩码；式(12)中正的$\Delta_{kj}^{\mathrm{cond}}$表示验证 MAE 上升。其含义是固定预测器依赖，不是成分干预的因果效应。

$$\Delta_{kj}^{\mathrm{cond}}=\mathrm{MAE}\left(y_{V,j},F_{j}\left(X_{V,-k},\tilde{x}_{k}\right)\right)-\mathrm{MAE}\left(y_{V,j},F_{j}(X_{V})\right)\tag{12}$$

式中，$\Delta_{kj}^{\mathrm{cond}}$为固定预测器的条件置换 MAE 增量，$X_{V}$与$y_{V,j}$为当前验证集的原测量输入及目标列；$V$表示外层验证样本集。$X_{V,-k}$保留其他输入，$\tilde{x}_{k}$按训练折条件方案替换字段且保留缺失掩码。正增量表示验证误差上升，描述预测依赖而不是干预因果。

总关联共识、分层自助重采样（bootstrap）稳定排名、风格校正及留一风格排名构成四路。对每个候选先比较 Top-10、Top-3 支持数，再用式(13)的倒数秩分数[8]H_{kj}排序，避免直接相加不同量纲的重要性。500 次 bootstrap 在风格内保留样本数，每次重算预处理与排名；式(14)以入选前三的次数衡量选择频率π_{kj}。另逐一删除 22 个风格检验关系是否由单风格主导。稳定选择与互补半样本的思想用于诊断可重复性和噪声入选[9][10]，这里不援引未满足假设的理论误选保证。

$$H_{kj}=\sum_{\ell=1}^{4}\frac{1}{60+r_{\ell,kj}}\tag{13}$$

式中，ℓ=1,…,4 分别索引总关联、分层自助、风格校正和留一风格四路证据，$r_{\ell,kj}$为字段k对目标j在该路中的正整数名次，H_{kj}为倒数秩融合分数，60 为预先固定的平滑常数。最终排序仍先比较既定支持数，再以该分数处理次序。

命题 6-1　同支持数下的秩支配。对同一感官目标，若两个候选的 Top-10、Top-3 支持数相同，且候选甲在四路的名次均不劣于候选乙，并至少一路严格更优，则式(13)的融合分数对候选甲更高。

证明。各路名次为正数，平滑常数 60 固定，倒数函数在正数域严格递减；名次逐路不劣使每项倒数不小，至少一路严格不等使总和严格增大。支持数相同时才比较该分数，故该候选不会在融合分数层被名次受支配者反超；支持数不同不适用。□

$$\pi_{kj} = \frac{1}{B}\sum_{b=1}^{B} \mathrm{I}\left(k \in \mathcal{S}_b^{(j)}\right) \tag{14}$$

式中，B=500 为风格内自助重抽次数，b为重抽索引，$\mathcal{S}_b^{(j)}$为第b次重抽后目标j的前三项严格单体字段集合，I(·)为示性函数，事件成立取 1 否则取 0。π_{kj}取 0-1，表示该字段在重复选择中进入前三的比例，不是因果概率。

统计排序固定后，再依据同名锚点、特异描述词、来源/老化/异味注释核验化学合理性。仅当校准浓度c_k和感知阈值τ_k具有相同字段、单位时，按式(15)计算无量纲气味活性值（OAV）；log10 存储值不直接除以阈值。A/B/C/D 分别表示直接特异证据较充分、类别证据较完整、至少一侧支持但特异性不足，以及弱预测或缺乏特异支持。$\mathrm{OAV} \geq 1$ 只说明达到单体阈值量级，混合协同、掩蔽与基质效应仍需实验。

题面缺陷项的可用阈值取自附件 8 的 Sensory thresholds 表 threshold_median 列：DMS 为 19 μg/L，双乙酰为 0.09 mg/L，4VG 为 0.20 mg/L；反-2-壬烯醛的该阈值缺失，记为 N/A，不以 0 替代。直接浓度不升与降至感知阈值以下是两个条件；已知源浓度时可检查前者，阈值缺失不能据此确认后者。

$$\mathrm{OAV}_k = \frac{c_k}{\tau_k}, \tau_k > 0 \tag{15}$$

式中，OAV_k为字段k的无量纲气味活性值，c_k为可核验的校准浓度，τ_k为同字段且同单位的正感知阈值。浓度及阈值单位不一致或字段身份不可比时不计算该比值，log10 存储值须先还原浓度。

6.2 模型求解

本文将稀疏与图正则回归转为增广 Lasso，用循环坐标下降求解；条件图消融采用 GraphicalLasso 的坐标下降模式并折内选参，模块用 Louvain 划分。四路按支持数与倒数秩排序；500 次分层自助和留一风格用于检验，最终预测器仅作独立条件归因。

分层 bootstrap 共 500 次，第 b 次使用种子 20261024+10000b（b=0,…,499）；各风格内部有放回抽取原样本数，再合并打乱。四路先比较 Top-10 与 Top-3 支持数，再比较式(13)的倒数秩和，平滑常数为 60，同分按最优单路名次及字段代码确定。CPSS 采用 100 对互补半样本：每个风格内随机排列后对半分，奇数多出的一款随配对编号奇偶轮换；第 p 对使用 NumPy SeedSequence([20261025,p])，p=0,…,99。负对照随机种子为 20261025+117000，包含 50 次风格内目标置换和 20 次影子重复，均在相应半样本内重新排序。

表 10 采用第 5 章相同外层划分比较候选方法。Spearman Top-3 基线的R^2为 0.1064，PLS/VIP 与 ElasticNet 约为 0.209；图正则R^2为 0.2084、MAE 为 0.3530，未形成额外预测收益。最终感官模型R^2/MAE/Spearman 为 0.2564/0.3361/0.4302；它承担独立归因，正式答案仍由四路排序决定。图 9 的描述网络在$|\rho| = 0.60$ 保留 264 条正边、2 条负边；图 10 将预测性能与四路排序 Jaccard 分开，后者不是预测精度。

表 10 相同外层划分下的候选模型与最终感官模型性能

方法	R^2（均值±SD）	MAE（均值±SD）	RMSE	Spearman
全局均值	-0.0081±0.0024	0.4173±0.0005	0.5690	-0.0576
Spearman Top-3	0.1064±0.0043	0.3843±0.0010	0.5273	0.2969
PLS/VIP	0.2089±0.0069	0.3556±0.0015	0.4890	0.4023
ElasticNet	0.2086±0.0046	0.3529±0.0012	0.4903	0.3917
有符号图 ElasticNet	0.2084±0.0047	0.3530±0.0013	0.4905	0.3919
ExtraTrees 条件归因	0.2359±0.0058	0.3422±0.0011	0.4794	0.4144
感官组收缩 NNLS	0.2564±0.0070	0.3361±0.0016	0.4714	0.4302

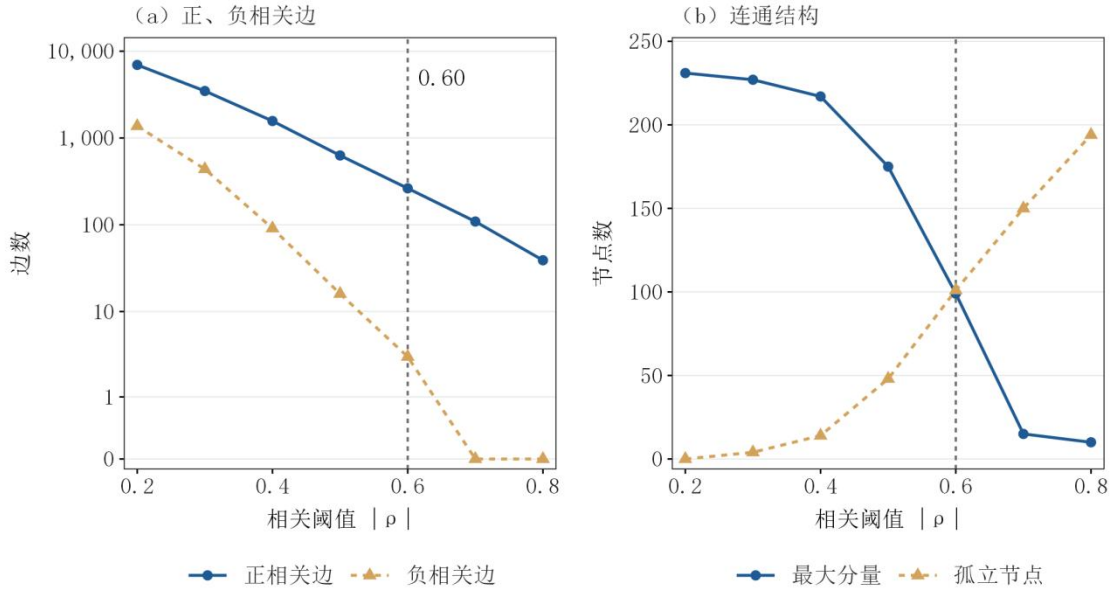


图 9 化合物共表达网络的阈值选择与结构敏感性

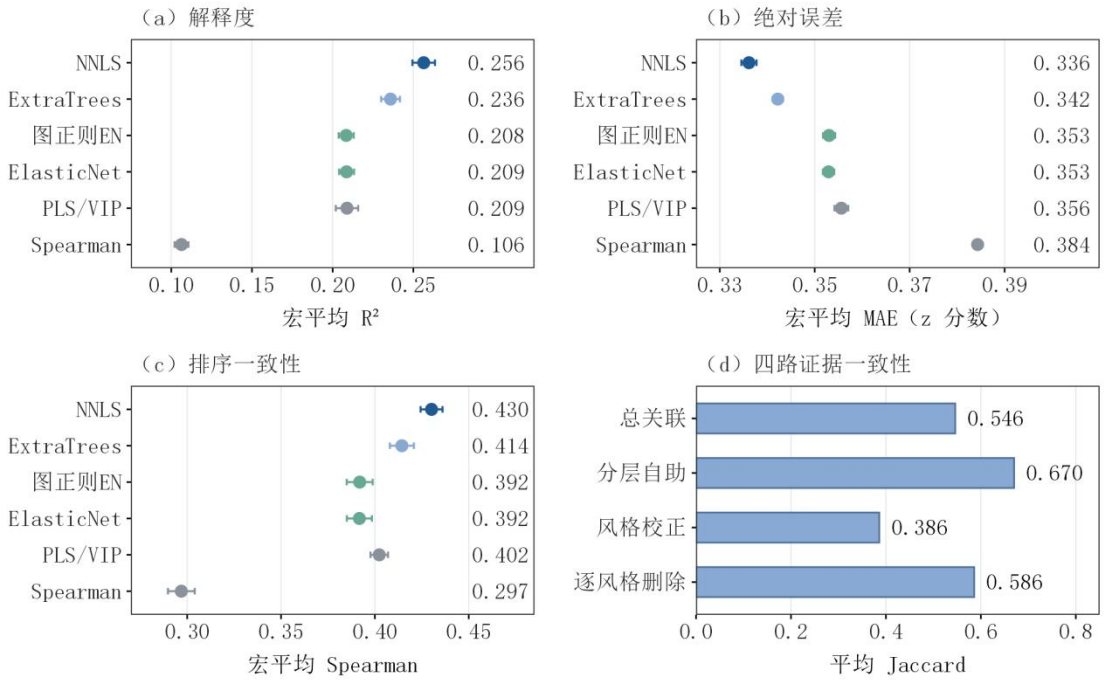


图 10 候选模型的折外性能与稳健排序的四路一致性

本问的正式答案是 50 个感官目标各 3 项、共 150 条化合物-感官关系。完整顺序及各对入选频率、留一风格保留率和化学等级列于“问题二_50 维 Top3 与证据.csv”；表 11 和图 12 仅展示其中 8 条 A 级关系，不代替完整答案。

按 bootstrap 频率选出的 150 对关系中，频率中位 0.428，43/31/23 条分别达到 0.60/0.70/0.80，留一风格均值 0.6582。正式四路融合 Top-3 同为 150 对，完整结果另随支撑材料提供，但频率中位为 0.395、40 条达到 0.60，留一风格均值/中位为 0.6076/0.6591。总关联参考集的均值 0.6406 对应另一集合，与 bootstrap 集仅重合 113 对，不能混用。bootstrap 两两 Top-3 平均 Jaccard 为 0.245；未校正与风格校正排序平均 Jaccard 为 0.264，仅 2/50 目标完全一致，表明共线替代和风格扰动仍影响名次。

图 11 按既有化学等级展示正式答案的样本与风格扰动稳定性。稳定共聚类得到 170 个模块，其中 155 个为单字段，最大模块 15 个字段；留一风格的 1100 个风格-目标组合平均 Jaccard 为 0.523。最终强/中/探索性稳健关系为 38/52/60 条；目标中 22 个$R^2 \geqslant 0.20$、12 个介于 0.10-0.20、16 个低于 0.10。弱目标仍按题面输出前三项，但只能解释为探索性关联。

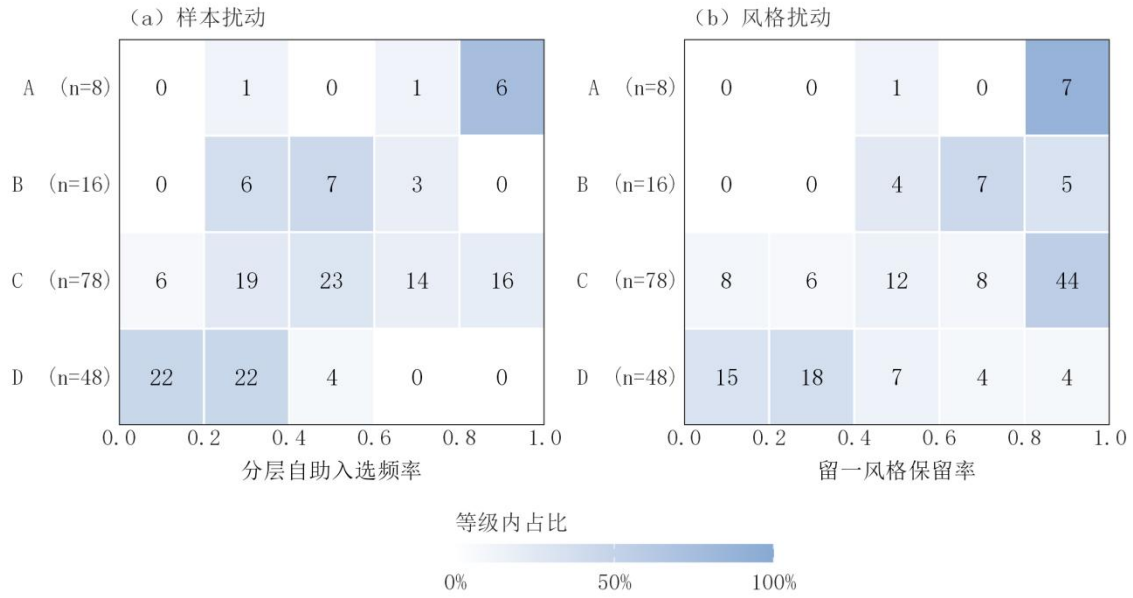


图 11 不同化学证据等级的样本与风格扰动稳定性

化学等级 A/B/C/D 共 8/16/78/48 条，73 条浓度与阈值可比，23 条中位 OAV ≥ 1。表 11 与图 12 列出 8 条 A 级关系，涵盖乙酸乙酯、4-乙烯基愈创木酚、乳酸、乙醇和二氧化碳。OAV 的 N/A 表示不可比，零表示零观测，二者不互相替代；分级只解释证据强弱，不改变统计排序。

表 11 化学证据较充分的代表性关键关系

感官维度	名次	关键化合物	自助入选频率	留一风格保留率	中位 OAV
指甲油样香气	1	乙酸乙酯	0.774	1.000	1.203
指甲油样风味	1	乙酸乙酯	0.878	1.000	1.203
4-乙烯基愈创木酚	1	4-乙烯基愈创木酚	0.992	1.000	0.000
酸度	1	乳酸	0.900	0.955	0.346
酒精感	1	乙醇	0.920	0.955	4.140
酒体	2	乙醇	0.812	0.955	4.140
碳酸感	3	二氧化碳	0.318	0.500	不可比
乳酸感	1	乳酸	0.992	1.000	0.346

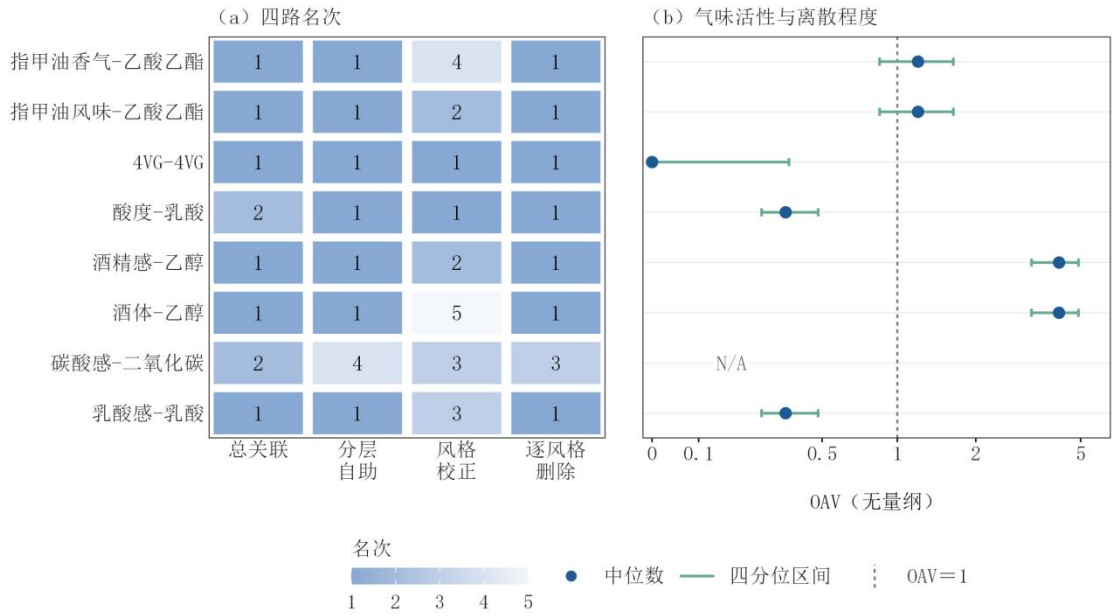


图 12 A 级化合物-感官关系的四路名次与气味活性值

6.3 模型检验与结果讨论

一致性核验得到 50×3 完整答案，同折最终预测器重建最大绝对误差 1.998×10^{-15}；模块与条件字段置换分别形成 13900、327000 条折级记录。验证观测不参与填补、尺度、构图、条件分组或风格均值估计。与 Shapley 加性解释（SHAP）所回答的加性归因问题不同[11]，这里的条件置换关注相关输入下的预测依赖，各方法的功能及采用边界汇总于表 12；文献任务与量纲不同，数字不直接横比。

表 12 方法家族与本文采用方式

方法与来源	本文用途	采用方式与边界
相关网络[5]	描述成对共变	全量图仅作描述；监督图在训练折重算，相关不等于因果。
稀疏条件图[6]	核验条件依赖	采用折内 GraphicalLasso 检验结构，不替换正式四路排序。
条件置换与 SHAP[7][11]	解释预测器依赖	本文采用模块和条件置换；SHAP 仅作文献比较。
稳定选择[9][10]	检验重采样稳定性	频率用于稳定性说明，不援引未满足假设的理论误选保证。
四路稳健排序（倒数秩项[8]）	综合四路统计证据	先比较支持数，再按倒数秩排序；预测归因独立报告。

把最终归因、偏相关和模块稳定性等权融合后，互补半样本稳定选择（CPSS）的频率中位、互补半样本 Jaccard 及留一风格中位生存率从 0.1425/0.1669/0.6591 降为 0.0750/0.1151/0.4091，并遗漏CO_2锚点。这一结构消融不支持替换四路答案，也不据此另建正式化学分级。统计稳健性决定排序，最终模型归因作为独立解释证据保留，二者结论不一致时不强行融合。

表 13 和图 13 报告负对照及直接锚点。目标置换在频率阈值 0.60/0.70/0.80/0.90 下均为 0 条，但 2000 条半样本记录的 6000 个前三槽中，影子占 1524（25.4%）、真实占 4476，每记录均值 0.762 与 2.238；不能称零误选。正式前三保留 4 对直接锚点，CO_2的 bootstrap 名次第 4、融合第 3；DMS、双乙酰、反-2-壬烯醛同名字段分别位于 bootstrap 第 29/92/61，未进入正式前三。

表 13 补充检验的数值结果与解释范围

检验项	结果	口径与解释
bootstrap 候选稳定性	中位 0.428；43/150≥0.60；留一风格均值 0.6582	按 bootstrap 频率选出的 150 对关系；集合对应后支持候选稳定性描述
最终融合稳定性	中位 0.395；40/150≥0.60；留一风格均值 0.6076、中位 0.6591	正式 150 对关系；尾部仍有替代与风格敏感性
目标置换	0/0/0/0 条	入选频率阈值 0.60/0.70/0.80/0.90；仅针对所列阈值
影子字段	真实 4,476 槽；影子 1,524 槽（25.4%）	2,000 条记录，共 6,000 个前三槽；真实字段占优，噪声入选非零
直接锚点	最终保留 4 对；CO_2 bootstrap 第 4、最终第 3	7 对单体锚点中 4 对保留；DMS/双乙酰/反-2-壬烯醛为 29/92/61
折内前三字段遮蔽	平均ΔMAE +0.0005806；中位 −0.00009657；22/50 退化	固定最终感官模型，附件感官标准化尺度；对象为同折条件效应前三，不是充分性
遮蔽后较大退化	酒精感+0.02244（+6.99%）；酒花总体+0.01143；苦度+0.00501	30 折分别计算，再按目标平均；不包含重训或重新选择字段的变化

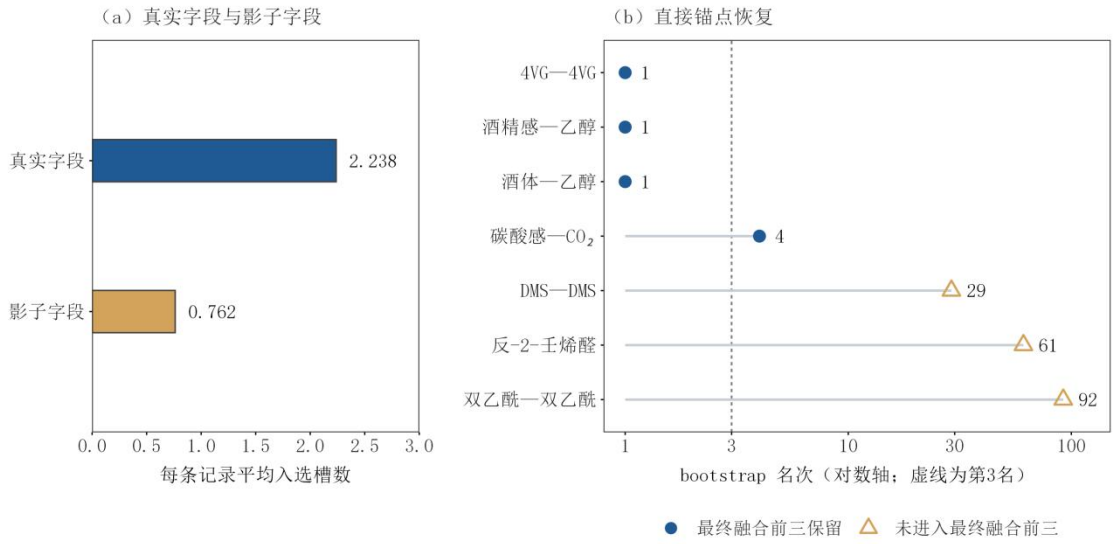


图 13 负对照、影子字段与直接锚点恢复

固定最终预测器，将同折条件效应前三字段替换为训练中位数后，50 目标平均ΔMAE 为+0.0005806，中位−0.00009657，22 个目标误差上升。酒精感增加 0.02244（6.99%），酒花总体和苦度增加 0.01143、0.00501。遮蔽集合不是正式融合 Top-3，故只能说明固定模型对折内重要字段的敏感性，不是正式答案的删除效应或充分性检验；该结果也不包含重新训练和重新选字段的不确定性。

七、 问题三的模型建立与求解

7.1 模型建立

本问条件标签为啤酒风格，Tripel、Kriek、Lambic 分别只有 28、10、11 款实测样本。为在高维下保持非正态边缘并限制依赖估计方差，采用经验边缘与收缩正态 Copula；生成、画像和迁移沿用第 5 章契约，但额外保留缺失、地板、校准零和参考冲突状态，见表 14。全量生成仅用于描述画像，外层评价的边缘、尺度和相关阵均只从训练折估计。

图 14 分开显示真实输入与既有接口完整性。Tripel、Kriek、Lambic 的实测样本为 28/10/11；中间面板来自附件 4 真实观测，点与横线为中位数及四分位距。每风格 300 谱对应 15000 个有限预测数值，总量 45000 表示 900 份画像的 50 维数值，不是新增 45000 个独立样本。

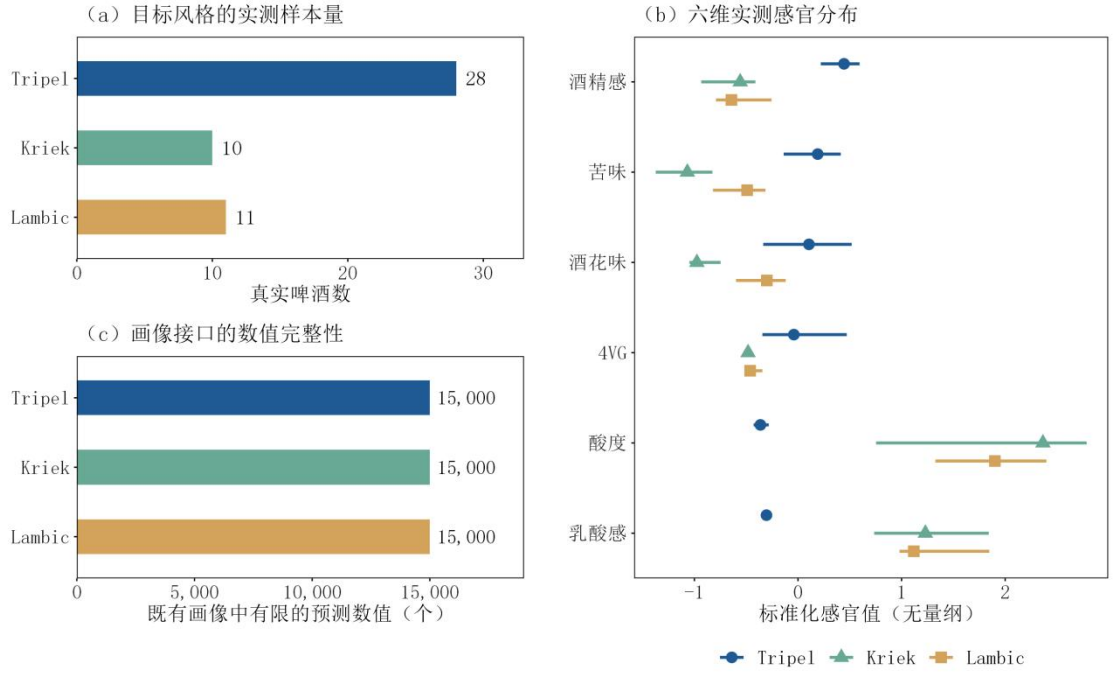


图 14 目标风格的真实样本、实测感官分布与画像接口完整性

表 14 问题三数据口径与处理规则

对象	处理规则	保留的边界
主键	按 beer 对齐化学谱、风格和感官数据	beer_id 不连接、不建模
化学字段	按字段核验结果保留 log10 或原尺度	不对所有字段统一指数化
缺失/地板	保留缺失掩码和地板状态	未知浓度不作为安全上限
感官画像	使用第一问最终感官组收缩 NNLS	0-6 仅作训练折经验映射
迁移约束	已知直接浓度逐项比较，未知项返回未知	未知不计入通过分母
校准零	保留原始/校准尺度的零状态及其来源	不把校准零直接认作真实无浓度
参考冲突	参考字段与输入不一致时保留冲突记录	不静默替换实测输入或认定安全

Copula 将边缘与依赖结构分离[12]。设风格 s 中的字段平均秩为$r_{ik}^{(s)}$、样本量为n_s，按式(16)映射到标准正态潜在变量u_{ik}，避免与第 5 章堆叠基预测 z 混用。潜在样本协方差Σ_s按训练折内 Ledoit-Wolf 准则向球形结构收缩[13]，如式(17)；D_s取收缩协方差的对角部分，由式(18)得到相关阵R_s。

$$u_{ik} = \Phi^{-1}\left(\frac{r_{ik}^{(s)} - 0.5}{n_s}\right) \tag{16}$$

式中，s为风格索引，n_s为当前训练集中该风格的真实样本数，$r_{ik}^{(s)}$为风格内字段k的平均秩；Φ为标准正态分布函数，Φ^{-1}为其反函数，u_{ik}为 Copula 的无量纲正态潜在坐标。恒定字段保留其状态，不由生成抽样增加真实信息。

$$\widehat{\Sigma}_s = (1-\alpha_s)\Sigma_s + \alpha_s \frac{\mathrm{tr}(\Sigma_s)}{p}\mathrm{I}_p \tag{17}$$

式中，Σ_s为当前风格潜在坐标的经验协方差，$\widehat{\Sigma}_s$为其 Ledoit-Wolf 收缩估计，α_s为训练数据确定的收缩强度；tr 为矩阵迹，I_p为p阶单位矩阵。球形收缩目标的平均方差由协方差矩阵迹除以字段维数得到。

$$R_s = D_s^{\frac{-1}{2}} \widehat{\Sigma}_s D_s^{\frac{-1}{2}} \tag{18}$$

式中，D_s为$\widehat{\Sigma}_s$对角元素构成的对角矩阵，$D_s^{\frac{-1}{2}}$逐对角取倒平方根，R_s为正态 Copula 的相关矩阵。近零方差作下界保护并保持矩阵对称半正定；该相关矩阵与迁移设计空间的精度矩阵分开估计。

在潜在相关正态分布采样后，以训练风格的经验分位函数Q_{sk}按式(19)反变换；恒定或地板字段保留其实际观测状态。各风格最终生成 300 份谱，再调用第 5 章感官接口形成标准化预测和 0-6 相对画像。画像的 10%-90%区间描述生成样本散布，不是均值置信区间或原始品评分数。

$$x_k^* = Q_{sk}\big(\Phi(u_k^*)\big) \tag{19}$$

式中，Q_{sk}为风格s、字段k的训练经验分位函数，采用线性分位插值；u_k^*为从相关正态分布抽样得到的潜在分量，x_k^*为对应生成谱的字段存储值。星号表示生成值，不表示该谱是唯一或最优方案。

迁移端点包括目标风格谱池与同分位端点。目标谱池每风格 2000 份，按设计尺度到源谱的距离选最近 256 份；另加入同分位端点，其第 k 个分量为$a_k = Q_{sk}\left(\mathcal{F}_{\mathrm{src,k}}\big(x_{0,\mathrm{k}}\big)\right)$。式中，$\mathcal{F}_{\mathrm{src,k}}$为源风格其余实测样本中小于源值的比例，加上等于源值比例的一半；Q_{sk}为目标风格同名字段的经验分位函数，采用线性分位插值。源啤酒不计入其自身参考分布。

迁移从源谱x_0沿目标候选端点 a 搜索，用$t \in [0,1]$表示路径系数，以区别于 NNLS 收缩η，路径见式(20)。对源浓度可核验的二甲基硫（DMS）、双乙酰、4VG、反-2-壬烯醛施加式(21)的不升约束；未知项返回未知，不能填补成安全上限。陈旧酒花味无单一可比化学列，由固定感官代理另行检验。

$$x(t) = (1-t)x_0 + ta, 0 \le t \le 1 \tag{20}$$

式中，x_0为源啤酒的原存储谱，a为目标风格候选端点，t为 0-1 的路径比例，$x(t)$为线性插值后的候选存储谱。已对数化字段在存储尺度插值，不等价于真实浓度或配方的线性混配。

$$c_k^*(t) = \min\Big(c_k\big(x(t)\big), c_k(x_0)\Big) \tag{21}$$

式中，$c_k(x)$表示按已核验的字段尺度将化学谱还原为直接浓度，$c_k^*(t)$为施加源浓度不升约束后的候选浓度。该式仅对源值、单位及尺度已知的直接缺陷字段使用；未知项保留未知，不由填补值构造安全上界。

命题 7-1　已知直接浓度的截断保序性。对身份、单位、尺度均可核验的缺陷字段，式(21)给出的候选浓度不高于源浓度；若源浓度不超过同单位感知阈值，则截断后亦不超过该阈值。

证明。两数的最小值不大于其中任一数，故$c_k^*(t)$不大于$c_k(x_0)$。若源浓度不超过阈值，传递性即得结论。源浓度已超阈时，该操作只保证不升，不能保证降至阈下；缺失或尺度未知字段、以及陈旧酒花味代理均不属于本命题。□

目标接近程度采用设计坐标中的 Mahalanobis 距离[14]，见式(22)；候选相对源谱的均方根改动见式(23)。两者均在固定设计参考尺度下计算，不等于酿造成本、工艺可实现性或实测迁移误差。

$$d_s(x)=\sqrt{\frac{\left(v(x)-\mu_s^{\text{design}}\right)^{\mathrm{T}}\Omega_s\left(v(x)-\mu_s^{\text{design}}\right)}{p}} \tag{22}$$

式中，$v(x)$为化学谱在固定设计参考尺度下的坐标，第k分量按$v_k(x)=\frac{x_k-\mu_{x,k}}{\sigma_{x,k}}$计算；$\mu_{x,k}$及$\sigma_{x,k}$分别为字段的设计参考均值及标准差。$\mu_s^{\text{design}}$为目标风格的设计坐标均值，$\Omega_s$为相同坐标下 Ledoit-Wolf 协方差的精度矩阵。$d_s(x)$是按p维归一化的 Mahalanobis 距离，包含平方根与维数分母；该矩阵不等同于 Copula 潜在相关矩阵R_s。

$$\Delta(x,x_0)=\sqrt{\frac{\sum_{k=1}^{p}\left(v_k(x)-v_k(x_0)\right)^2}{p}} \tag{23}$$

式中，$\Delta(x,x_0)$为候选相对源谱在相同设计坐标中的均方根改动，$v_k(x)$和$v_k(x_0)$分别为候选和源的第k个标准化坐标。该值无量纲，反映谱的相对改动，不表示原浓度差、酿造成本或真实工艺误差。

7.2 模型求解

本文用经验秩变换及 Ledoit-Wolf 闭式收缩估计依赖结构，经相关正态采样与经验分位反变换生成谱。迁移对固定端点和七个路径比例作有限枚举，施加直接浓度约束后，按可行性与改动量排序；其最优性仅限给定候选集。

生成验证的 5 个基础种子为 20261025、20261026、20261027、20261028、20261029；第 r 次重复、第 f 折、第 s 个风格的实际种子为基础种子+1000r+100f+10s，均从 0 编号，风格顺序为 Tripel、Kriek、Lambic。每折留出数及每个种子生成数分别为 9-10、3-4、3-4，相应训练风格样本为 18-19、6-7、7-8，均非单点。距离与覆盖先在各折、各种子中计算，再求重复内均值、风格等权均值及重复间标准差，不跨折池化后计算距离。式(17)的收缩强度由各训练风格的 Ledoit-Wolf 估计自动确定，不设跨风格统一常数。

迁移另使用每风格 2000 份目标谱池，种子为 20261025+s；每组采用上述 256 个邻近端点及 1 个同分位端点。路径比例为 0.05、0.10、0.20、0.35、0.50、0.75、1.00。主预测器搜索要求目标相对收益至少 0.01，代理变化的均值和 90%分位均不高于 −0.01，支持重复比例至少 0.9。选定最小改动候选后，再由两个预测模型检验零裕量不恶化；表 19 的 0.01/0.02 裕量是固定候选的后续敏感性检验，不等同于初选约束。

生成方法复用 10×3 外层划分，每折每风格 5 个生成种子，生成数匹配真实留出数；折与种子先平均，再对风格等权和重复汇总。表 15 和图 15 显示收缩 Copula 在训练折分位稳健尺度下的边缘一阶 Wasserstein 距离（W1）、联合能量距离（energy）、几何覆盖和对数离散误差为 0.4762/0.5813/0.5580/0.2475。整行局部混合的 W1/energy/覆盖为 0.4740/0.5772/0.6599，部分指标较优，但离散误差增至 0.2925；PCA 及基函数邻域混合亦有覆盖与离散的权衡。综合选择继续保留 Copula，不能概括为所有变体每项都更差。

表 15 生成方法的稳健尺度比较

方法	W1 ↓	energy ↓	覆盖率 ↑	离散误差 ↓
风格收缩 Copula	0.4762	0.5813	0.5580	0.2475
整行局部混合	0.4740	0.5772	0.6599	0.2925
PCA 邻域混合	0.4768	0.5805	0.6660	0.3026
基函数邻域混合	0.4786	0.5834	0.6580	0.3014

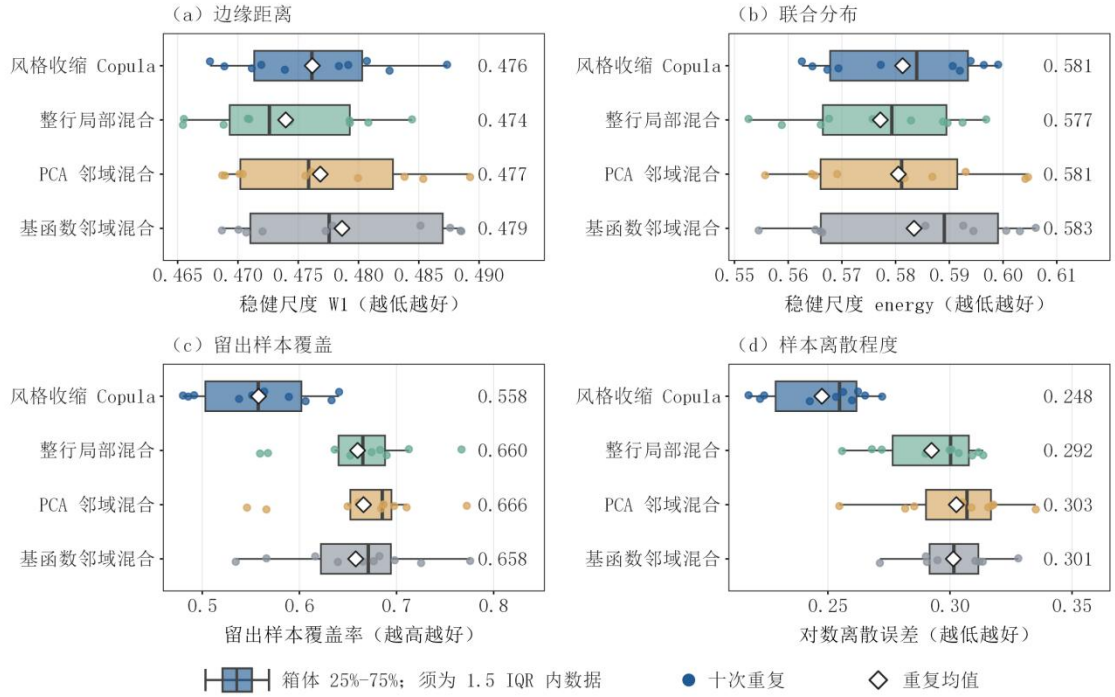


图 15 风格条件生成方法在外层重复验证中的分布保真与多样性

表 16 和图 16 给出三风格六个预定维度的画像。Tripel 的酒精感、苦味、酒花味和 4VG 相对较高，Kriek/Lambic 的酸度与乳酸感较高。画像为同一预测接口下的相对比较；例如 4VG 的外层R^2约 0.255，不能因生成画像分离就认为真实品评差异已获验证。

表 16 三种风格生成样本的六维相对画像均值

维度	Tripel	Kriek	Lambic
酒精感	4.4	0.9	1.1
苦味	3.9	0.6	1.1
酒花味	3.7	0.8	1.6
4VG	4.6	0.8	1.3
酸度	3.0	5.6	5.6
乳酸感	2.9	5.6	5.7

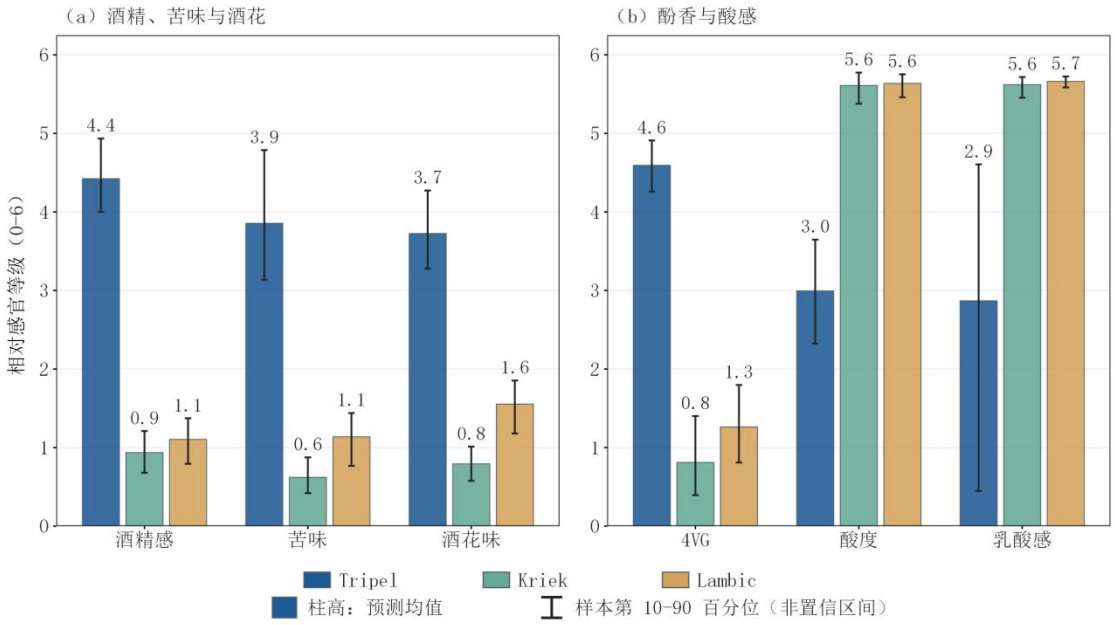


图 16 三目标风格生成样本的相对感官画像

表 16 中 4VG 的 4.6 是感官相对等级，不能与 0.20 mg/L 的化学阈值直接比较。图 14 的实测感官分布也显示 Tripel 相对较高，但风格特征不免除 4VG 的缺陷核验；附件 8 的校准零存在解释不确定性，不能据此认定真实浓度为 0 或不存在越阈。

按题面缺陷名单，DMS、双乙酰、4VG 及反-2-壬烯醛在直接浓度可比时逐项核对源值、候选值和阈值；陈旧酒花味无单一可比化学列，另用固定感官代理。已知源值不升与超阈状态分开报告，未知项不计为通过，完整风格画像和迁移核验表随文提供。

固定最小改动规则用于 11 个源与 3 个目标组成的 33 组任务，29 组得到有谱候选。核验后 2 条为待实测统计候选、26 条直接证据不足、1 条代理不稳、4 组无搜索可行谱。两条统计候选均来自同一源啤酒 230，分别指向 Tripel 与 Kriek，目标距离下降 38.36%和 37.36%，RMS 改动为 0.4862 和 0.5822，见表 17 与图 17。结果说明约束下存在统计设计点，尚不足以形成实际酿造方案。

表 17 两条待实测统计候选的迁移结果

目标风格	目标距离下降	谱 RMS 改动	代理模型状态	化学状态
Tripel	38.36%	0.4862	两模型预测不升	仍有待核验状态
Kriek	37.36%	0.5822	两模型预测不升	仍有待核验状态

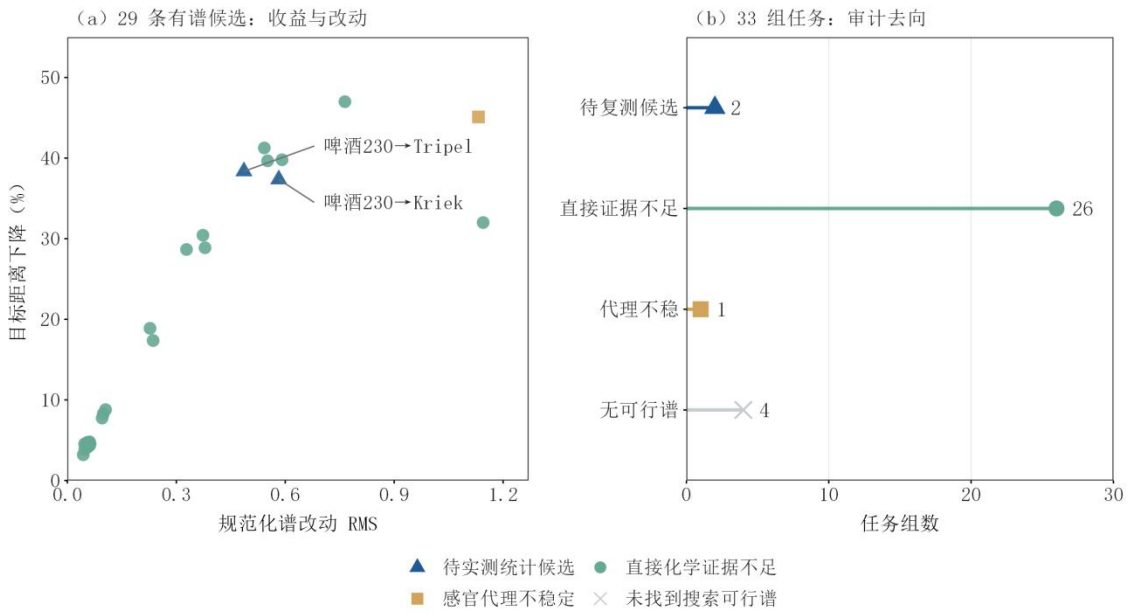


图 17 跨风格迁移候选的目标接近程度与证据状态

7.3 模型检验与结果讨论

完整性检验覆盖 450 组生成缓存和 900 份描述谱，45000 个感官值全部有限；训练、验证啤酒无交叉，近训练复制率及整组塌缩标记均为 0，完整谱重构误差约10^{-12}。原已恒定的检测地板字段不视为塌缩。表 18 区分经验基线、协方差收缩、局部结构和约束搜索的功能；表 19 将程序一致性、尺度、分母与未检范围并列。

表 18 生成与迁移方法的功能对照及消融结论

方法家族	回答的问题	本文对应结果	采用边界
经验边缘与 Copula	如何保持非正态边缘和联合依赖	收缩 Copula 作为主生成方法	目标风格样本稀疏时依赖收缩
协方差收缩	如何稳定估计高维相关结构	风格内协方差向对角结构收缩	不把收缩后的相关解释为机理
局部混合/低秩结构	如何扩大覆盖或降维采样	作为消融，覆盖提升伴随离散误差	不能只凭覆盖率替换主模型
Mahalanobis 约束搜索	如何度量目标接近与限制谱改动	用于迁移候选排序	候选须经过直接化学核验
强化学习或深度生成	如何学习长期动作或复杂生成策略	当前数据缺少动作-状态转移和独立实测	登记为后续研究，不进入本问主结果

原始训练尺度的 W1/energy 为 1.0773/6.3933，分位稳健尺度为 0.4762/0.5813，说明极端记录影响显著；稳健评价先将训练、留出和生成数据截断至训练折 1%-99%分位，再除以截断训练数据的字段标准差（小于10^{-12}取 1），不回写原观测。Tripel/Kriek/Lambic 稳健 W1 分别 0.4114/0.5684/0.4487，覆盖 0.3356/0.7389/0.5994。几何覆盖采用上述尺度下除以$\sqrt{p}$的欧氏距离：真实留出点到最近生成点的距离不超过该点到其他真实留出点的最近邻半径时记为覆盖；它既不是预测区间覆盖也不是安全率；小风格相关 RMSE 较大，只作描述。

表 19 问题三补充检验与适用范围

检验项	结果	口径与结论
完整性与划分	30 折×3 风格×5 种子共 450 组齐全，生成数逐组匹配留出数；900 份画像×50 目标全部有限	训练与验证 beer 无交叉；支持程序一致性和画像完整性
复制与退化	近训练复制率 0，整组塌缩标记 0	复制按规范化 RMS$\leqslant 10^{-6}$；实测本已恒定的地板维度保留
尺度敏感性	原尺度 W1/energy=1.0773/6.3933；稳健尺度 0.4762/0.5813	稳健边界仅来自训练折 1%-99%分位；极端记录影响显著
逐风格一致性	Tripel/Kriek/Lambic 稳健 W1 为 0.4114/0.5684/0.4487；覆盖 0.3356/0.7389/0.5994	风格等权；小风格相关 RMSE 较大，仅作描述
保持边缘的独立置换	W1 变化 0；energy 差+0.00109，相关 RMSE 差+0.00851	差为置换−原谱；五次置换先平均；Kriek 的 energy 差为−0.00009，证据不一致
候选分母核验	891 候选=844 有谱+47 结构拒绝；844=512 直接证据不足+205 无收益+112 拒绝+15 待代理	293 条为非支配候选，与上述分类存在重叠；15 条因代理信息缺失暂不可判定
固定迁移与裕量	33 任务中 2 统计候选/26 证据不足/1 代理不稳/4 无谱；裕量 0/0.01/0.02 个标准化感官单位对应 2/0/0	固定候选不重新选择；两候选来自同一源，需独立实测
未验证范围	未见风格泛化、独立批次和实际酿造尚未验证	已知风格内留出；不支持可实际酿造、唯一或全局最优主张

结构消融按实际方法名呈现于表 20：共享低秩、风格残差收缩、风格残差与地板状态的相关 RMSE 略降，但 W1 及 energy 增加，energy 相对 Copula 增加 0.0658-0.0703。改善单一相关诊断未带来联合分布收益。其他神经残差、地板感知边缘及局部迁移尝试也未形成稳定的综合增量，保留其比较而不增加模型复杂度。

表 20 共享结构变体的生成消融检验

方法	W1 / energy	覆盖率 / 相关 RMSE
风格收缩 Copula	0.4762 / 0.5813	0.5580 / 0.7748
共享低秩	0.5648 / 0.6516	0.5070 / 0.7706
风格残差收缩	0.5643 / 0.6471	0.4989 / 0.7723
风格残差与地板状态	0.5646 / 0.6471	0.4992 / 0.7723

在固定缓存内逐字段独立置换生成行，450 组各重复 5 次，共 2250 次，边缘 W1 按构造不变。图 18 显示置换−原谱的等权 energy 差+0.00109、相关 RMSE 差+0.00851，三风格覆盖均下降；Tripel 的 energy 差+0.00275，而 Kriek 为−0.00009。联合配对获得有限且不均匀的支持，不能声称所有风格的依赖均准确恢复。图中标准差只描述重复划分和随机扰动，种子、折及置换不是独立实测样本。

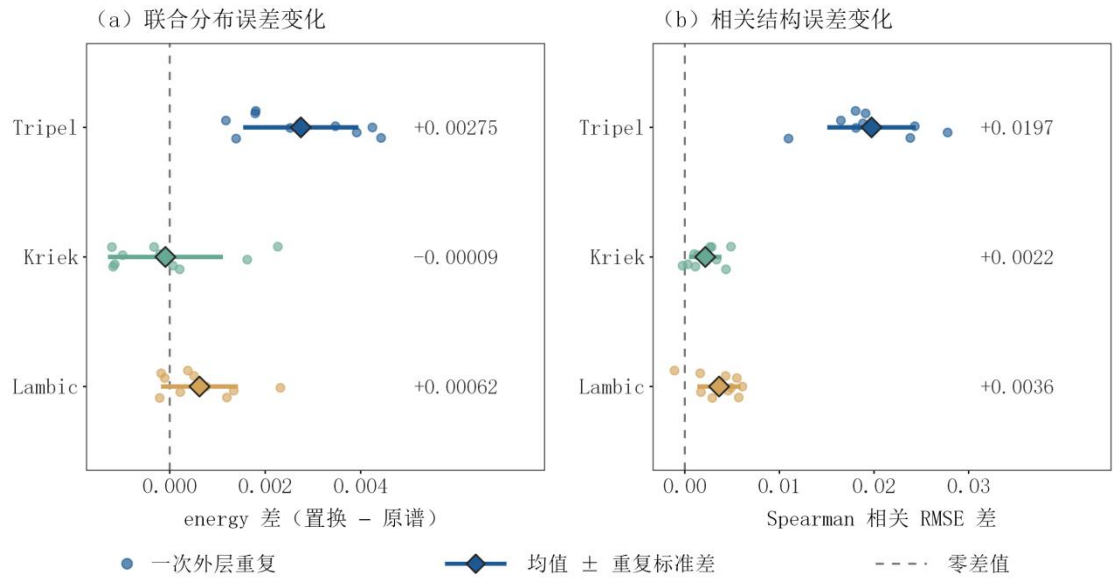


图 18 保持边缘分布的独立置换下三种风格的联合结构检验

候选池 891 条中 844 条形成化合物谱，其中 512 条直接证据不足、205 条无收益、112 条越界被拒；另 15 条因代理信息缺失暂无法判定，不计为失败。后续 33 项迁移任务的两条统计候选，在增加 0.01 或 0.02 个标准化感官单位的双模型下降裕量后均不再满足，说明结论接近代理误差边界。陈旧酒花味代理按 10 次逐重复指标取均值时，R^2约 0.03、MAE 约 0.30；图 5(c)先平均 10 次折外预测再评价，分别约为 0.06 和 0.29，二者口径不同。预测不升不能直接替代实测改善。

八、 模型评价、改进与推广

8.1 模型的优点

（1）数据与接口统一。三问使用一致的主键、尺度和折外接口，预测、统计排序与生成检验各有对应证据，便于逐层核对。

（2）兼顾互补性与稳定性。感官组收缩利用四类学习器互补，在保留不利排序指标的同时改善连续解释度；四路排名同时检验样本扰动和风格变化。

（3）结论边界明确。化学核验区分预测依赖与缺陷证据；生成方法联合考察边缘、联合、覆盖和离散，并保留候选未知状态。

8.2 模型的缺点

（1）数据与量表受限。样本量及风格不均衡限制弱目标和小风格推断，聚合感官信号无法恢复原始评委评分。

（2）区间缺少独立验证。重复验证共享样本，残差校准为样本内检查；新增小规模重训缺逐样本预测，无法完整分解训练不确定性。

（3）前三项存在替代。变量共线、影子字段入选非零，遮蔽检验不能证明正式答案充分。

（4）生成与迁移尚待实测。画像及迁移依赖预测器，未见风格、独立批次和真实酿造尚未验证；两条候选同源且对微小裕量敏感，不能据此作安全或工艺结论。

8.3 模型的改进与推广

后续应优先补充原评委评分、检测限、独立批次和加标品评，使量表、删失及化学机理能够分别验证；对弱目标先核对测量与候选资格，再考虑扩大模型。外部样本

用于独立校准区间和跨批次评估，保存每次重训的逐样本预测以分解训练不确定性。若取得配方、工艺和动作-状态数据，可把统计迁移扩展为受实验反馈约束的顺序设计；在此之前，本流程适用于同一测量体系下的画像筛查和待实测候选排序。

参考文献

[1] OpenAI. ChatGPT: GPT-5.6 Sol [EB/OL]. 使用日期: 2026-10-27[2026-10-28]. https://chatgpt.com/.

[2] 深度求索（DeepSeek）. DeepSeek: DeepSeek-V4.1-Flash [EB/OL]. 使用日期: 2026-10-27[2026-10-28]. https://www.deepseek.com/.

[3] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees [J]. Machine Learning, 2006, 63 (1): 3-42. DOI:10.1007/s10994-006-6226-1.

[4] Breiman L. Stacked regressions [J]. Machine Learning, 1996, 24 (1): 49-64. DOI:10.1023/A:1018046112532.

[5] Zhang B, Horvath S. A general framework for weighted gene co-expression network analysis [J]. Statistical Applications in Genetics and Molecular Biology, 2005, 4 (1): Article 17. DOI:10.2202/1544-6115.1128.

[6] Friedman J, Hastie T, Tibshirani R. Sparse inverse covariance estimation with the graphical lasso [J]. Biostatistics, 2008, 9 (3): 432-441. DOI:10.1093/biostatistics/kxm045.

[7] Strobl C, Boulesteix A L, Kneib T, et al. Conditional variable importance for random forests [J]. BMC Bioinformatics, 2008, 9 (1): 307. DOI:10.1186/1471-2105-9-307.

[8] Cormack G V, Clarke C L A, Buettcher S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods [C]//Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval. 2009: 758-759. DOI:10.1145/1571941.1572114.

[9] Meinshausen N, Bühlmann P. Stability selection [J]. Journal of the Royal Statistical Society: Series B, 2010, 72 (4): 417-473. DOI:10.1111/j.1467-9868.2010.00740.x.

[10] Shah R D, Samworth R J. Variable selection with error control: another look at stability selection [J]. Journal of the Royal Statistical Society: Series B, 2013, 75 (1): 55-80. DOI:10.1111/j.1467-9868.2011.01034.x.

[11] Lundberg S M, Lee S I. A unified approach to interpreting model predictions [C]//Advances in Neural Information Processing Systems 30. 2017: 4765-4774.

[12] Sklar A. Fonctions de répartition à n dimensions et leurs marges [J]. Publications de l'Institut de Statistique de l'Université de Paris, 1959, 8: 229-231.

[13] Ledoit O, Wolf M. A well-conditioned estimator for large-dimensional covariance matrices [J]. Journal of Multivariate Analysis, 2004, 88 (2): 365-411. DOI:10.1016/S0047-259X(03)00096-4.

[14] Mahalanobis P C. On the generalized distance in statistics [J]. Proceedings of the National Institute of Sciences of India, 1936, 2 (1): 49-55.

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具[1][2]，用于辅助赛题理解与任务拆解、术语规范与语言润色、公式排版、图表标题与参考文献格式检查、模型优点与局限的梳理、图表结果表述的量化修改，以及附录与支撑材料结构整理。模型假设、算法选择、参数设定、程序运行、结果判断与最终文字均由参赛队逐项复核，详细使用情况见支撑材料《AI 工具使用详情说明.pdf》。

附录

附录 A 支撑材料文件列表

支撑材料压缩包命名为“A2601605 附件.zip”。表 A.1 列出文件组成，表 A.2 列出 30 个核心程序及运行说明；所列路径均相对于“A2601605 附件”文件夹。

表 A.1 支撑材料文件列表

序号	文件名称	格式	主要内容
1	A2601605 源程序\	PY / R	三问核心模型、求解、检验与绘图程序，共 30 个文件（20 个 PY、10 个 R）
2	A2601605 数据_不含赛题原始数据\	CSV / JSON	字段映射、固定划分、候选字典、数据契约与预处理记录，共 18 个文件
3	A2601605 结果\	CSV / GZ / NPZ / JSON	三问正式结果、模型检验与绘图数据，共 111 个文件
4	A2601605 论文图表\	PNG	正文图 1-18 及附图 S1-S3，共 21 张图片
5	A2601605 软件与运行说明.txt	TXT	软件版本、运行命令及 Python、R 环境快照
6	AI 工具使用详情说明	PDF	AI 工具使用详情另行补充，未计入现有附件包

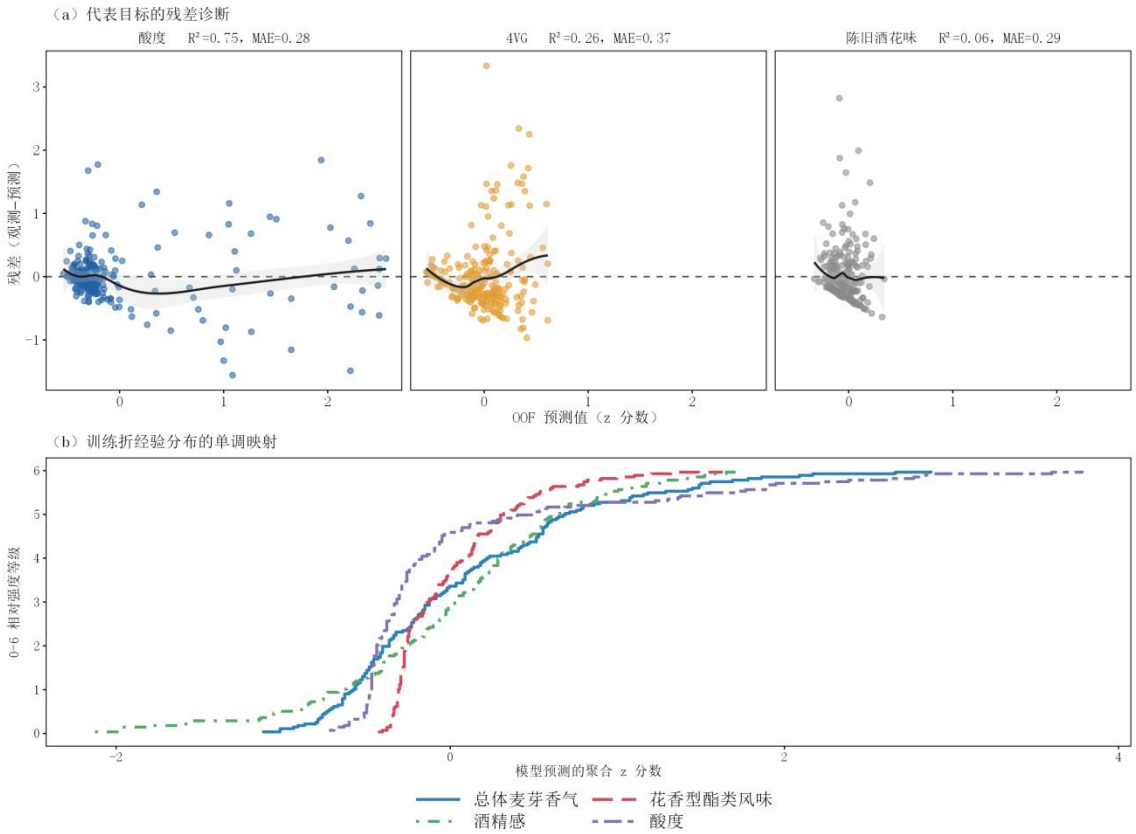
表 A.2 程序文件与支撑材料对照表

序号	支撑材料相对路径	所属问题
1	A2601605 源程序\Q1_01_基础学习器.py	问题一
2	A2601605 源程序\Q1_02_梯度提升学习器.py	问题一
3	A2601605 源程序\Q1_03_感官组收缩 NNLS.py	问题一
4	A2601605 源程序\Q1_04_嵌套折内求解.py	问题一
5	A2601605 源程序\Q1_05_预测评价.py	问题一
6	A2601605 源程序\Q1_06_感官预测运行.py	问题一
7	A2601605 源程序\Q1_07_主图.R	问题一
8	A2601605 源程序\Q1_08_检验图.R	问题一
9	A2601605 源程序\Q1_09_附图排版.R	问题一
10	A2601605 源程序\Q2_01_候选选择与图正则.py	问题二
11	A2601605 源程序\Q2_02_条件置换与模块.py	问题二
12	A2601605 源程序\Q2_03_重采样稳定性.py	问题二
13	A2601605 源程序\Q2_04_风格效应校正.py	问题二
14	A2601605 源程序\Q2_05_四路稳健排序.py	问题二
15	A2601605 源程序\Q2_06_冻结感官模型.py	问题二
16	A2601605 源程序\Q2_07_最终模型条件归因.py	问题二
17	A2601605 源程序\Q2_08_化学核验与 OAV.py	问题二
18	A2601605 源程序\Q2_09_主图.R	问题二
19	A2601605 源程序\Q2_10_负对照与锚点图.R	问题二
20	A2601605 源程序\Q3_01_风格收缩 Copula.py	问题三
21	A2601605 源程序\Q3_02_浓度转换与缺陷约束.py	问题三
22	A2601605 源程序\Q3_03_受约束迁移求解.py	问题三
23	A2601605 源程序\Q3_04_感官画像推理.py	问题三
24	A2601605 源程序\Q3_05_迁移证据状态.py	问题三
25	A2601605 源程序\Q3_06_分布一致性检验.py	问题三
26	A2601605 源程序\Q3_07_生成与迁移图.R	问题三
27	A2601605 源程序\Q3_08_联合结构检验图.R	问题三
28	A2601605 源程序\技术路线.R	公共绘图
29	A2601605 源程序\数据概览.R	公共绘图
30	A2601605 源程序\绘图主题.R	公共绘图
31	A2601605 软件与运行说明.txt	全文

附录 B 补充图片

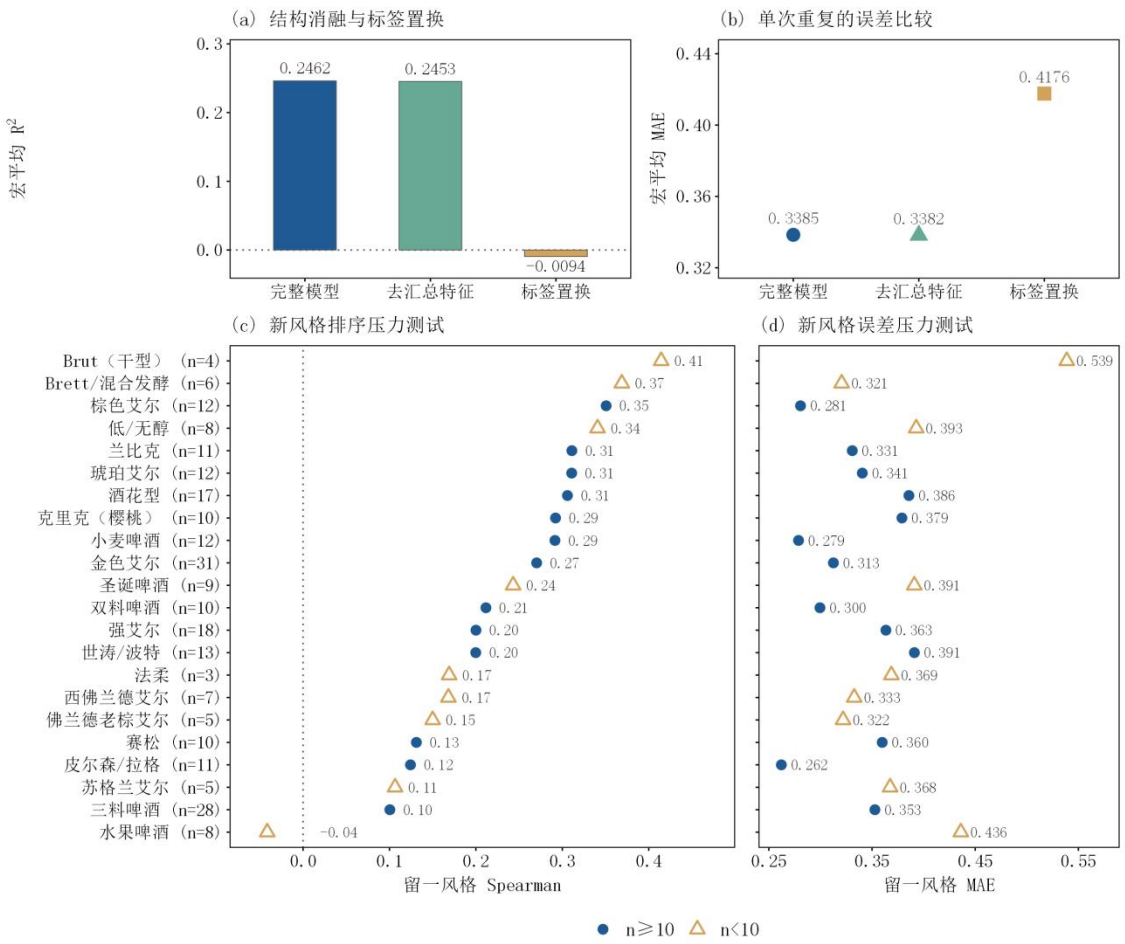
B.1 问题一补充图表

附图 S1 补充正文相对量表映射与误差形态的解释。左侧检查固定代表目标的残差，右侧展示训练经验分布将连续预测转为 0-6 相对强度；它不恢复原始品评分，也不提供单个新样本预测区间。



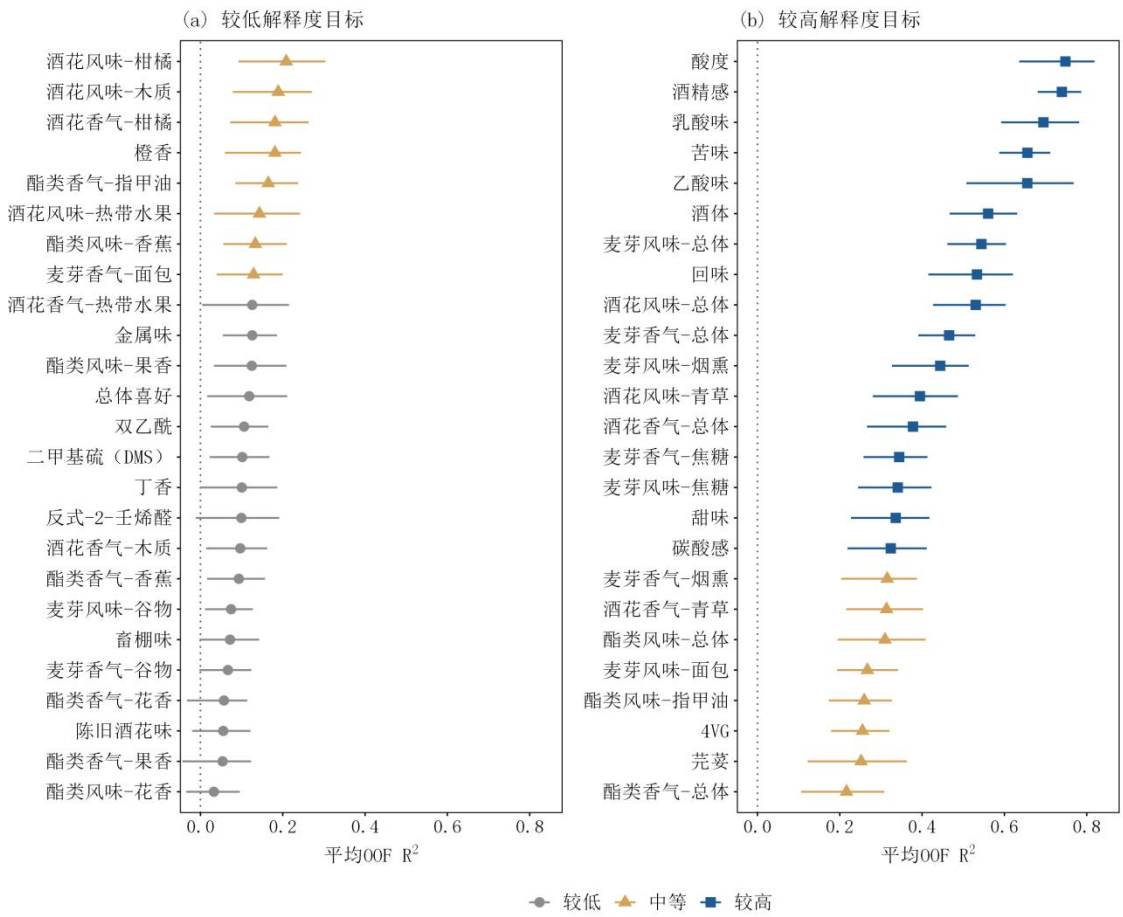
附图 S1 固定目标残差与相对量表映射

附图 S2 补充正文结构消融和压力检验。MAE 面板采用点图；目标置换为全局一次啤酒目标行置换后固定交叉验证，留一风格仅以 Spearman 和 MAE 描述。重训未保存逐样本预测，因此不能据汇总值分解重新训练不确定性。



附图 S2 结构消融、标签置换与留一风格检验

附图 S3 补充正文 50 个目标的相对可靠性，中文名称逐项对应真实目标代码。两栏按固定顺序展示解释度和排序结果，三层是相对分组而非固定可用性标准；小风格与弱目标不据此获得独立泛化保证。



附图 S3 逐目标折外预测可靠性

附录 C 问题一模型核心程序

本附录列出问题一感官组收缩 NNLS 与训练折内求解的核心函数节选，完整源程序见表 A.2。以下命令在解压后的 A2601605 附件目录执行。

C.1 感官组收缩 NNLS（Q1_03_感官组收缩 NNLS.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q1_03_感官组收缩 NNLS.py"。

节选自 Q1_03_感官组收缩 NNLS.py 第 4-71 行。

```
from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.optimize import nnls


@dataclass(frozen=True)
class NNLSStackingFit:
    weights: np.ndarray
    intercept: np.ndarray
    global_weights: np.ndarray
    scope: str
    shrinkage: float

    def predict(self, stack) -> np.ndarray:
        values = np.asarray(stack, dtype=float)
        if values.ndim != 3 or values.shape[1:] != self.weights.shape:
            raise ValueError("prediction tensor must have the fitted
targets/models shape")
        if not np.isfinite(values).all():
            raise ValueError("prediction tensor must be finite")
        return np.einsum("ntm,tm->nt", values, self.weights) + self.intercept


def _solve(centered_stack, centered_y, scales) -> np.ndarray:
    design = (centered_stack / scales[None, :, None]).reshape(-1,
centered_stack.shape[2])
    response = (centered_y / scales[None, :]).reshape(-1)
    if not np.any(design):
        return np.zeros(design.shape[1], dtype=float)
    weights, _ = nnls(design, response, maxiter=max(100, design.shape[1] * 30))
    return weights
```

```
def fit_nnls_stacking(stack, y, *, groups=None, scope="shared", shrinkage=.5)
-> NNLSStackingFit:
    """Fit on outer-training inner-OOF predictions and their training labels
only.

    Weights are nonnegative but need not sum to one. Target centering removes
    intercepts before NNLS, so an intercept is free to be negative afterward.
    Standardizing each training target gives shared/group fits equal target
units.
    """
    predictions, truth = np.asarray(stack, dtype=float), np.asarray(y,
dtype=float)
    if (predictions.ndim != 3 or min(predictions.shape) < 1
            or predictions.shape[0] < 3 or truth.shape != predictions.shape[:2]
            or not np.isfinite(predictions).all() or not
np.isfinite(truth).all()):
        raise ValueError("finite samples-by-targets-by-models predictions must
align with y")
    if scope not in {"shared", "group", "target"}:
        raise ValueError("scope must be shared, group or target")
    if not np.isfinite(shrinkage) or not 0 <= shrinkage <= 1:
        raise ValueError("shrinkage must be in [0, 1]")
    if scope == "group" and (groups is None or len(groups) != truth.shape[1]):
        raise ValueError("group labels must cover all registered targets")
    means = predictions.mean(axis=0)
    target_means = truth.mean(axis=0)
    centered, y_centered = predictions - means, truth - target_means
    scales = np.std(truth, axis=0)
    scales = np.where(scales > 1e-12, scales, 1.)
    global_weights = _solve(centered, y_centered, scales)
    weights = np.tile(global_weights, (truth.shape[1], 1))
    if scope != "shared":
        labels = np.arange(truth.shape[1]) if scope == "target" else
np.asarray(groups, dtype=str)
        for label in dict.fromkeys(labels.tolist()):
            positions = np.flatnonzero(labels == label)
            local = _solve(centered[:, positions], y_centered[:, positions],
scales[positions])
            weights[positions] = (1. - shrinkage) * local + shrinkage *
global_weights
    intercept = target_means - np.einsum("tm,tm->t", means, weights)
    return NNLSStackingFit(weights.copy(), intercept.copy(),
global_weights.copy(),
                           str(scope), 1. if scope == "shared" else
float(shrinkage))
```

C.2 训练折内 OOF 与堆叠求解（Q1_04_嵌套折内求解.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q1_04_嵌套折内求解.py"。

节选自 Q1_04_嵌套折内求解.py 第 89-132 行。

```
def fit_outer_nnls(x_train, y_train, x_validation, *, base_config, groups,
                   scopes, shrinkage, inner_splits, seed) -> dict:
    """Rebuild base-model OOF entirely inside outer training, then fit NNLS."""
    y = np.asarray(y_train, dtype=float)
    splits = make_inner_splits(len(y), int(inner_splits), int(seed))
    assignment = np.full(len(y), -1, dtype=int)
    for fold, (_, valid) in enumerate(splits):
        assignment[valid] = fold
    inner_predictions, outer_predictions, selections = [], [], {}
    for model_position, model in enumerate(base_config["base_models"]):
        candidate_rows = []
        for candidate_position, parameter in enumerate(_candidates(base_config,
model)):
            prediction = np.full_like(y, np.nan)
            for fold, (train, valid) in enumerate(splits):
                prediction[valid] = _fit_base(
                    model, parameter, x_train[train], y[train], x_train[valid],
base_config,
                    random_state=int(seed) + model_position * 100000 +
candidate_position * 1000 + fold,
                )
            if not np.isfinite(prediction).all():
                raise RuntimeError("stacking base inner OOF is incomplete")
            candidate_rows.append({"parameter": parameter, "prediction":
prediction,
                                   "metrics": matrix_metrics(y, prediction)})
        selected = min(candidate_rows, key=lambda row:
_parameter_key(row["metrics"]))
        inner_predictions.append(selected["prediction"])
        outer_predictions.append(_fit_base(
            model, selected["parameter"], x_train, y, x_validation, base_config,
            random_state=int(seed) + model_position * 100000,
        ))
        selections[model] = {
            "parameter": selected["parameter"],
            "candidate_metrics": [{"parameter": row["parameter"], "metrics":
row["metrics"]}
                                  for row in candidate_rows],
        }
    inner_stack, outer_stack = np.stack(inner_predictions, axis=-1),
np.stack(outer_predictions, axis=-1)
```

```
    predictions, parameters = {}, {}
    for scope in scopes:
        fitted = fit_nnls_stacking(inner_stack, y, groups=groups, scope=scope,
shrinkage=shrinkage)
        predictions[str(scope)] = fitted.predict(outer_stack)
        parameters[str(scope)] = {"weights": fitted.weights.tolist(),
                                  "intercept": fitted.intercept.tolist(),
                                  "global_weights":
fitted.global_weights.tolist()}
    return {"predictions": predictions, "parameters": parameters,
"base_selection": selections,
            "inner_fold_assignment": assignment, "inner_prediction_stack":
inner_stack,
            "outer_base_stack": outer_stack}
```

附录 D 问题二模型核心程序

本附录列出问题二稳健排序、条件归因与化学核验的核心函数节选，完整源程序见表 A.2。以下命令在解压后的 A2601605 附件目录执行。

D.1 完整名次适配与倒数秩融合（Q2_05_四路稳健排序.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q2_05_四路稳健排序.py"。

节选自 Q2_05_四路稳健排序.py 第 5-135 行。

```
from __future__ import annotations
import numpy as np
import pandas as pd


def reciprocal_rank_consensus(
    rankings: pd.DataFrame,
    *,
    expected_methods: int,
    rrf_k: float = 60.0,
) -> pd.DataFrame:
    """Fuse complete method rankings with support-aware reciprocal rank
fusion."""
    required = {"method", "target_code", "feature_code", "aggregate_rank"}
    missing = required - set(rankings.columns)
    if missing:
        raise ValueError(f"rankings miss columns: {sorted(missing)}")
    if int(expected_methods) < 2 or float(rrf_k) <= 0:
        raise ValueError("expected_methods must be >=2 and rrf_k must be
positive")
    table = rankings[list(required)].copy()
    if table["method"].nunique() != int(expected_methods):
        raise ValueError("method count does not match expected_methods")
    duplicated = table.duplicated(["method", "target_code", "feature_code"])
    if duplicated.any():
        raise ValueError("each method-target-feature row must be unique")
    coverage = table.groupby(["target_code",
"feature_code"])["method"].nunique()
    if not coverage.eq(int(expected_methods)).all():
        raise ValueError("every target-feature pair must cover every method")

    table["rrf_component"] = 1.0 / (float(rrf_k) + table["aggregate_rank"])
    table["is_top3"] = table["aggregate_rank"].le(3)
    table["is_top10"] = table["aggregate_rank"].le(10)
    result = (
        table.groupby(["target_code", "feature_code"], sort=False)
        .agg(
```

```
            rrf_score=("rrf_component", "sum"),
            method_top3_support=("is_top3", "sum"),
            method_top10_support=("is_top10", "sum"),
            best_method_rank=("aggregate_rank", "min"),
            mean_method_rank=("aggregate_rank", "mean"),
            median_method_rank=("aggregate_rank", "median"),
        )
        .reset_index()
    )
    result = result.sort_values(
        [
            "target_code",
            "method_top10_support",
            "method_top3_support",
            "rrf_score",
            "best_method_rank",
            "feature_code",
        ],
        ascending=[True, False, False, False, True, True],
        kind="mergesort",
    ).reset_index(drop=True)
    result["consensus_rank"] = result.groupby("target_code").cumcount() + 1
    ranks = table.pivot(
        index=["target_code", "feature_code"],
        columns="method",
        values="aggregate_rank",
    ).reset_index()
    ranks.columns = [
        column if column in ("target_code", "feature_code") else
f"rank__{column}"
        for column in ranks.columns
    ]
    result = result.merge(
        ranks, on=["target_code", "feature_code"], how="left",
validate="one_to_one"
    )
    return result


def aggregate_leave_one_style_ranks(
    rankings: pd.DataFrame,
    *,
    expected_styles: int,
    rrf_k: float = 60.0,
) -> pd.DataFrame:
```

```
    """Aggregate complete leave-one-style ranks into one rank per
target/feature."""
    required = {"omitted_style", "target_code", "feature_code",
"consensus_rank"}
    missing = required - set(rankings.columns)
    if missing:
        raise ValueError(f"rankings miss columns: {sorted(missing)}")
    if int(expected_styles) < 2 or float(rrf_k) <= 0:
        raise ValueError("expected_styles must be >=2 and rrf_k must be
positive")
    table = rankings[list(required)].copy()
    if table["omitted_style"].nunique() != int(expected_styles):
        raise ValueError("observed style count does not match expected_styles")
    coverage = table.groupby(["target_code", "feature_code"])[
        "omitted_style"
    ].nunique()
    if not coverage.eq(int(expected_styles)).all():
        raise ValueError("every target-feature pair must cover every omitted
style")
    table["is_top3"] = table["consensus_rank"].le(3)
    table["is_top10"] = table["consensus_rank"].le(10)
    table["rrf_component"] = 1.0 / (float(rrf_k) + table["consensus_rank"])
    result = (
        table.groupby(["target_code", "feature_code"], sort=False)
        .agg(
            top3_survival_frequency=("is_top3", "mean"),
            top10_survival_frequency=("is_top10", "mean"),
            median_rank=("consensus_rank", "median"),
            rank_q25=("consensus_rank", lambda values:
float(values.quantile(0.25))),
            rank_q75=("consensus_rank", lambda values:
float(values.quantile(0.75))),
            mean_rrf=("rrf_component", "mean"),
        )
        .reset_index()
    )
    result["rank_iqr"] = result["rank_q75"] - result["rank_q25"]
    result = result.sort_values(
        [
            "target_code",
            "top3_survival_frequency",
            "top10_survival_frequency",
            "mean_rrf",
            "median_rank",
            "feature_code",
        ],
```

```
        ascending=[True, False, False, False, True, True],
        kind="mergesort",
    ).reset_index(drop=True)
    result["aggregate_rank"] = result.groupby("target_code").cumcount() + 1
    return result


def _as_lens(
    frame: pd.DataFrame, rank_column: str, method: str
) -> pd.DataFrame:
    result = frame[["target_code", "feature_code", rank_column]].copy()
    result = result.rename(columns={rank_column: "aggregate_rank"})
    result.insert(0, "method", method)
    return result
```

D.2 四路统计证据连接（Q2_05_四路稳健排序.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q2_05_四路稳健排序.py"。

节选自 Q2_05_四路稳健排序.py 第 138-149 行。

```
def solve_four_lens_consensus(total, bootstrap, adjusted, deletion_folds):
    """正式四路秩融合；输入为总关联、重采样、风格校正、留一风格结果。"""
    deletion = aggregate_leave_one_style_ranks(deletion_folds,
expected_styles=22)
    lenses = pd.concat([
        _as_lens(total, "consensus_rank", "total_association"),
        _as_lens(bootstrap, "bootstrap_rank", "bootstrap_stability"),
        _as_lens(adjusted, "consensus_rank", "style_adjusted"),
        _as_lens(deletion, "aggregate_rank", "leave_one_style_survival"),
    ], ignore_index=True)
    return reciprocal_rank_consensus(lenses, expected_methods=4,
rrf_k=60).rename(
        columns={"method_top3_support": "lens_top3_support",
                 "method_top10_support": "lens_top10_support"})
```

D.3 固定感官模型与模块置换（Q2_06_冻结感官模型.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q2_06_冻结感官模型.py"。

节选自 Q2_06_冻结感官模型.py 第 3-47 行。

```
from __future__ import annotations
import numpy as np
import pandas as pd


class FrozenFoldNNLS:
    """A four-learner raw-input predictor with already-shrunk frozen
weights."""
```

```
    def __init__(self,models,weights,intercept,feature_names):
        self.models=list(models);self.weights=np.asarray(weights,float).copy()

self.intercept=np.asarray(intercept,float).copy();self.feature_names=tuple(feat
ure_names)
        if (len(self.models)!=4 or self.weights.ndim!=2 or
self.weights.shape[1]!=4
                or self.intercept.shape!=(self.weights.shape[0],) or
(self.weights<0).any()
                or not np.isfinite(self.weights).all() or not
np.isfinite(self.intercept).all()):
            raise ValueError('invalid frozen weights or free target intercepts')
        self.weights.setflags(write=False);self.intercept.setflags(write=False)

    def predict(self,values):
        if isinstance(values,pd.DataFrame) and
tuple(values.columns.astype(str))!=self.feature_names:
            raise ValueError('raw feature order differs from frozen Q1')
        x=np.asarray(values,float)
        if x.ndim!=2 or x.shape[1]!=len(self.feature_names) or
np.isinf(x).any():
            raise ValueError('predict expects the complete ordered raw feature
matrix')
        stack=np.stack([np.asarray(model.predict(x),float) for model in
self.models],axis=-1)
        if stack.shape[1:]!=self.weights.shape or not np.isfinite(stack).all():
            raise ValueError('base predictions disagree with frozen target/model
dimensions')
        return np.einsum('ntm,tm->nt',stack,self.weights)+self.intercept


def
raw_module_permutation(model,values,module_codes,*,strict_feature_codes,rng):
    """Move raw module values and masks together, holding other fields fixed.

    The other 13 fields remain conditioning context. This is a predictive
    permutation diagnostic, not a chemically coherent causal intervention.
    """
    names=list(model.feature_names);members=list(module_codes)
    if not members or len(set(members))!=len(members) or not
set(members)<=set(strict_feature_codes):
        raise ValueError('module members must be distinct strict compound
fields')
    if isinstance(values,pd.DataFrame) and
tuple(values.columns.astype(str))!=tuple(names):
```

```
        raise ValueError('raw feature order differs from frozen Q1')
    x=np.asarray(values,float)
    if x.ndim!=2 or x.shape[1]!=len(names) or not len(x):raise
ValueError('complete raw input required')
    positions=[names.index(name) for name in members]

order=rng.permutation(len(x));changed=x.copy();changed[:,positions]=x[order][:,
positions]
    return model.predict(changed),order
```

D.4 条件残差置换与误差变化（Q2_07_最终模型条件归因.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q2_07_最终模型条件归因.py"。

节选自 Q2_07_最终模型条件归因.py 第 4-127 行。

```
from __future__ import annotations
from dataclasses import dataclass
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
try:
    from .Q2_06_冻结感官模型 import raw_module_permutation
except ImportError:
    from Q2_06_冻结感官模型 import raw_module_permutation


@dataclass
class ConditionalFeatureModel:
    pipeline: Pipeline
    n_conditioners: int

    def predict(self, conditioners, style) -> np.ndarray:
        numeric = np.asarray(conditioners, dtype=float)
        styles = np.asarray(style, dtype=str)
        if numeric.ndim != 2 or numeric.shape[1] != self.n_conditioners:
            raise ValueError("conditioner shape disagrees with fitted model")
        if styles.shape != (len(numeric),):
            raise ValueError("style vector must align with conditioners")
        frame = pd.DataFrame(
            numeric,
            columns=[f"x_{position}" for position in
range(self.n_conditioners)],
        )
```

```
        frame["style"] = styles
        return np.asarray(self.pipeline.predict(frame), dtype=float)


def fit_conditional_feature_model(
    conditioners, feature, style
) -> ConditionalFeatureModel:
    """Fit compact HGB E[X_j | module peers, style] on training rows only."""
    numeric = np.asarray(conditioners, dtype=float)
    target = np.asarray(feature, dtype=float)
    styles = np.asarray(style, dtype=str)
    if numeric.ndim != 2 or numeric.shape[0] < 20 or numeric.shape[1] < 1:
        raise ValueError("conditioners must contain at least 20 rows and one
column")
    if target.shape != (len(numeric),) or styles.shape != (len(numeric),):
        raise ValueError("feature and style must align with conditioners")
    if not np.isfinite(target).all():
        raise ValueError("conditional feature target must be finite")
    frame = pd.DataFrame(
        numeric,
        columns=[f"x_{position}" for position in range(numeric.shape[1])],
    )
    frame["style"] = styles
    numeric_columns = list(frame.columns[:-1])
    transformer = ColumnTransformer(
        [
            ("numeric", SimpleImputer(strategy="median"), numeric_columns),
            (
                "style",
                OneHotEncoder(handle_unknown="ignore", sparse_output=False),
                ["style"],
            ),
        ]
    )
    pipeline = Pipeline(
        [
            ("transform", transformer),
            (
                "model",
                HistGradientBoostingRegressor(
                    max_leaf_nodes=7,
                    min_samples_leaf=20,
                    learning_rate=0.05,
                    l2_regularization=1.0,
                    max_iter=100,
                    random_state=20261025,
```

```
                ),
            ),
        ]
    )
    pipeline.fit(frame, target)
    return ConditionalFeatureModel(pipeline, numeric.shape[1])


def residual_permutation(
    conditional_mean, residual, rng: np.random.Generator
) -> np.ndarray:
    """Keep the conditional mean fixed and permute only residual variation."""
    mean = np.asarray(conditional_mean, dtype=float)
    error = np.asarray(residual, dtype=float)
    if mean.ndim != 1 or error.shape != mean.shape or not len(mean):
        raise ValueError("conditional mean and residual must be aligned
vectors")
    if not np.isfinite(mean).all() or not np.isfinite(error).all():
        raise ValueError("conditional mean and residual must be finite")
    return mean + error[rng.permutation(len(error))]


def
_fold_field_rows(model,x_train,x_valid,y_valid,style_train,style_valid,modules,
feature_codes,target_codes,repeat,fold,seed,repeats=10):
    rows=[];feature_position={v:i for i,v in enumerate(model.feature_names)}
    baseline=model.predict(x_valid);base_mae=np.mean(np.abs(y_valid-
baseline),axis=0)
    strict_position={v:i for i,v in enumerate(feature_codes)}
    for module_id,members in enumerate(modules,1):
        selected=[feature_position[v] for v in members]
        # Module-level permutation is raw-input and keeps missing values
attached.
        for permutation_repeat in range(repeats):

changed_prediction,order=raw_module_permutation(model,x_valid,members,strict_fe
ature_codes=feature_codes,rng=np.random.default_rng(seed+module_id*1000+permuta
tion_repeat))
            delta=np.mean(np.abs(y_valid-changed_prediction),axis=0)-base_mae
            for target_position,target in
enumerate(target_codes):rows.append({'repeat':repeat,'fold':fold,'module_id':f'
M{module_id:03d}','module_size':len(members),'module_members':'|'.join(members)
,'target_code':target,'permutation_repeat':permutation_repeat,'delta_mae':delta
[target_position],'module_source':'fold_local_all_modules_no_validation_gate'})
        for feature in members:
```

```
            pos=feature_position[feature];peers=[feature_position[v] for v in
members if v!=feature]

train_feature=x_train.iloc[:,pos].to_numpy(float);observed=np.isfinite(train_fe
ature)

valid_feature=x_valid.iloc[:,pos].to_numpy(float);valid_observed=np.isfinite(va
lid_feature)
            if len(peers) and observed.sum()>=20 and valid_observed.any():

cond_imputer=SimpleImputer(strategy='median',keep_empty_features=True)

train_cond=cond_imputer.fit_transform(x_train.iloc[:,peers].to_numpy(float))

valid_cond=cond_imputer.transform(x_valid.iloc[:,peers].to_numpy(float))

cond=fit_conditional_feature_model(train_cond[observed],train_feature[observed]
,style_train[observed])

valid_pos=np.flatnonzero(valid_observed);conditional=cond.predict(valid_cond[va
lid_pos],style_valid[valid_pos])
                residual=valid_feature[valid_pos]-conditional
                for permutation_repeat in range(repeats):

changed=x_valid.copy();changed_values=valid_feature.copy();changed_values[valid
_pos]=residual_permutation(conditional,residual,np.random.default_rng(seed+pos*
1000+permutation_repeat));changed.iloc[:,pos]=changed_values
                    delta=np.mean(np.abs(y_valid-model.predict(changed)),axis=0)-
base_mae
                    for target_position,target in
enumerate(target_codes):rows.append({'repeat':repeat,'fold':fold,'module_id':f'
M{module_id:03d}','module_size':len(members),'module_members':'|'.join(members)
,'feature_code':feature,'target_code':target,'permutation_repeat':permutation_r
epeat,'delta_mae':delta[target_position],'module_source':'fold_local_all_module
s_no_validation_gate','conditional_scope':'observed_only_missing_mask_preserved
'})
    return pd.DataFrame(rows)
```

D.5 OAV 与化学证据核验（Q2_08_化学核验与 OAV.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q2_08_化学核验与 OAV.py"。

节选自 Q2_08_化学核验与 OAV.py 第 3-67 行。

```
from __future__ import annotations
import numpy as np
import pandas as pd
```

```
def _oav_summary(feature: str, threshold: float, calibrated: pd.DataFrame) ->
dict:
    empty = {
        "oav_n": 0,
        "raw_concentration_median": np.nan,
        "oav_q25": np.nan,
        "oav_median": np.nan,
        "oav_q75": np.nan,
        "oav_max": np.nan,
        "oav_above_1_fraction": np.nan,
        "oav_evidence": "unavailable",
    }
    if feature not in calibrated.columns or not np.isfinite(threshold) or
threshold <= 0:
        return empty
    values = pd.to_numeric(calibrated[feature],
errors="coerce").dropna().to_numpy(float)
    if not len(values):
        return empty
    oav = values / float(threshold)
    median = float(np.median(oav))
    fraction = float(np.mean(oav >= 1.0))
    if median >= 1.0:
        evidence = "median_above_threshold"
    elif fraction >= 0.25:
        evidence = "subset_above_threshold"
    else:
        evidence = "mostly_below_threshold"
    return {
        "oav_n": int(len(values)),
        "raw_concentration_median": float(np.median(values)),
        "oav_q25": float(np.quantile(oav, 0.25)),
        "oav_median": median,
        "oav_q75": float(np.quantile(oav, 0.75)),
        "oav_max": float(np.max(oav)),
        "oav_above_1_fraction": fraction,
        "oav_evidence": evidence,
    }


def _grade(row: pd.Series) -> str:
    semantic = int(row["semantic_evidence_level"])
    statistical = str(row["statistical_robustness_tier"])
    prediction = str(row["predictability_tier"])
    if (
```

```
    semantic == 4
    and statistical in {"R1_strong", "R2_moderate"}
    and prediction != "P3_weak"
) or (
    semantic == 3
    and statistical == "R1_strong"
    and prediction != "P3_weak"
):
    return "A"
if (
    semantic >= 2
    and statistical in {"R1_strong", "R2_moderate"}
    and prediction != "P3_weak"
) or (semantic >= 3 and statistical in {"R1_strong", "R2_moderate"}):
    return "B"
if semantic >= 1 or statistical in {"R1_strong", "R2_moderate"}:
    return "C"
return "D"
```

附录 E 问题三模型核心程序

本附录列出问题三风格生成、画像预测与约束迁移的核心函数节选，完整源程序见表 A.2。以下命令在解压后的 A2601605 附件目录执行。

E.1 风格收缩 Copula 与经验反变换（Q3_01_风格收缩 Copula.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q3_01_风格收缩 Copula.py"。

节选自 Q3_01_风格收缩 Copula.py 第 7-145 行。

```
import numpy as np
from scipy.stats import norm, rankdata
from sklearn.covariance import LedoitWolf


def _psd_factor(matrix):
    """Return a symmetric PSD matrix and a cached sampling factor."""
    symmetric=(np.asarray(matrix,float)+np.asarray(matrix,float).T)/2.
    eigenvalues,eigenvectors=np.linalg.eigh(symmetric)
    eigenvalues=np.clip(eigenvalues,0.,None)
    psd=(eigenvectors*eigenvalues)@eigenvectors.T
    psd=(psd+psd.T)/2.
    return psd,eigenvectors*np.sqrt(eigenvalues)


def _correlation_from_covariance(covariance):
    covariance=np.asarray(covariance,float)
    scale=np.sqrt(np.clip(np.diag(covariance),1e-12,None))
    correlation=covariance/np.outer(scale,scale)
    correlation=np.nan_to_num(correlation,nan=0.,posinf=0.,neginf=0.)
    correlation=(correlation+correlation.T)/2.
    np.fill_diagonal(correlation,1.)
    correlation,_=_psd_factor(correlation)
    scale=np.sqrt(np.clip(np.diag(correlation),1e-12,None))
    correlation=correlation/np.outer(scale,scale)
    correlation=(correlation+correlation.T)/2.
    np.fill_diagonal(correlation,1.)
    return correlation


class StyleGaussianGenerator:
    def fit(self,x,styles,*,shrinkage='ledoit_wolf',mean_shrinkage=.25):
        values=np.asarray(x,float);labels=np.asarray(styles,str)
        if values.ndim!=2 or labels.shape!=(len(values),) or len(values)<2 or
not np.isfinite(values).all():
            raise ValueError('finite training matrix and labels required')
        if shrinkage!='ledoit_wolf':
```

```
            raise ValueError('only registered Ledoit-Wolf covariance is
supported')
        if not 0.<=float(mean_shrinkage)<=1.:
            raise ValueError('mean_shrinkage must be in [0, 1]')
        self.global_mean=values.mean(0)
        self.global_scale=np.std(values,axis=0)
        self.global_scale=np.where(self.global_scale>1e-12,self.global_scale,1.)
        standardized=(values-self.global_mean)/self.global_scale
        self.models={};self.n_features=values.shape[1]
        for style in np.unique(labels):
            block=values[labels==style]
            block_standardized=standardized[labels==style]
            if len(block)<2:
                raise ValueError('each style needs at least two training rows')

            # Raw covariance is retained only to reproduce the immutable pilot.
            raw_estimator=LedoitWolf().fit(block)
            raw_mean=(1-
mean_shrinkage)*block.mean(0)+mean_shrinkage*self.global_mean

            estimator=LedoitWolf().fit(block_standardized)
            standardized_mean=(1-mean_shrinkage)*block_standardized.mean(0)

gaussian_covariance,gaussian_factor=_psd_factor(estimator.covariance_)

            latent=np.zeros_like(block_standardized)
            for column in range(self.n_features):
                if np.ptp(block[:,column])<=1e-12:
                    continue
                ranks=rankdata(block[:,column],method='average')
                latent[:,column]=norm.ppf((ranks-.5)/len(block))
            latent_estimator=LedoitWolf().fit(latent)

copula_covariance=_correlation_from_covariance(latent_estimator.covariance_)
            copula_covariance,copula_factor=_psd_factor(copula_covariance)

            self.models[str(style)]={
                'mean':raw_mean,
                'cov':raw_estimator.covariance_,
                'standardized_mean':standardized_mean,
                'gaussian_covariance':gaussian_covariance,
                'gaussian_factor':gaussian_factor,
                'copula_covariance':copula_covariance,
                'copula_factor':copula_factor,
                'rows':len(block),
                'style_values':block.copy(),
```

```
                'lower':block.min(0),
                'upper':block.max(0),
            }
        return self

    def diagnostics(self,style):
        if str(style) not in self.models:
            raise ValueError('unseen style')
        model=self.models[str(style)]
        return {

'gaussian_min_eigenvalue':float(np.linalg.eigvalsh(model['gaussian_covariance']
).min()),

'copula_min_eigenvalue':float(np.linalg.eigvalsh(model['copula_covariance']).mi
n()),
            'n_training_style':int(model['rows']),
        }

    def generate(self,style,n,*,method='style_gaussian',seed=20261025):
        if str(style) not in self.models:
            raise ValueError('unseen style')
        if n<1:
            raise ValueError('positive sample count required')
        model=self.models[str(style)];rng=np.random.default_rng(seed)
        pre_clip_cell_fraction=0.;pre_clip_row_fraction=0.
        if method=='style_gaussian':

values=rng.multivariate_normal(model['mean'],model['cov'],size=n,check_valid='w
arn')
        elif method=='style_gaussian_clipped':

latent=rng.normal(size=(n,self.n_features))@model['gaussian_factor'].T

raw=(latent+model['standardized_mean'])*self.global_scale+self.global_mean
            outside=(raw<model['lower'])|(raw>model['upper'])
            pre_clip_cell_fraction=float(outside.mean())
            pre_clip_row_fraction=float(outside.any(axis=1).mean())
            values=np.clip(raw,model['lower'],model['upper'])
        elif method=='global_gaussian':

values=rng.multivariate_normal(self.global_mean,np.eye(self.n_features),size=n,
check_valid='warn')
        elif method in {'style_copula','style_copula_shrunk'}:
            block=model['style_values']
            if method=='style_copula':
```

```
                z=np.empty_like(block)
                for column in range(self.n_features):
                    order=np.argsort(block[:,column],kind='stable')
                    ranks=np.empty(len(block));ranks[order]=np.arange(len(block))
                    z[:,column]=norm.ppf((ranks+.5)/len(block))
                corr=np.corrcoef(z,rowvar=False);corr=np.nan_to_num(corr,nan=0.)
                corr=(corr+corr.T)/2.;np.fill_diagonal(corr,1.)

latent=rng.multivariate_normal(np.zeros(self.n_features),corr,size=n,check_vali
d='warn')
            else:

latent=rng.normal(size=(n,self.n_features))@model['copula_factor'].T
            values=np.empty_like(latent)
            for column in range(self.n_features):
                sorted_values=np.sort(block[:,column])
                probabilities=np.clip(norm.cdf(latent[:,column]),0.,1.)

values[:,column]=np.quantile(sorted_values,probabilities,method='linear')
        else:
            raise ValueError('unsupported Gaussian baseline')
        return {
            'values':values,
            'style':str(style),
            'method':method,
            'n_training_style':model['rows'],
            'pre_clip_cell_fraction':pre_clip_cell_fraction,
            'pre_clip_row_fraction':pre_clip_row_fraction,
        }
```

E.2 感官画像推理与支持域检查（Q3_04_感官画像推理.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q3_04_感官画像推理.py"。

节选自 Q3_04_感官画像推理.py 第 7-104 行。

```
from __future__ import annotations
import numpy as np
import pandas as pd
from scipy.spatial.distance import cdist


class FrozenFoldNNLS:
    """A four-learner raw-input predictor with already-shrunk frozen
weights."""

    def __init__(self,models,weights,intercept,feature_names):
        self.models=list(models);self.weights=np.asarray(weights,float).copy()
```

```
self.intercept=np.asarray(intercept,float).copy();self.feature_names=tuple(feat
ure_names)
        if (len(self.models)!=4 or self.weights.ndim!=2 or
self.weights.shape[1]!=4
                or self.intercept.shape!=(self.weights.shape[0],) or
(self.weights<0).any()
                or not np.isfinite(self.weights).all() or not
np.isfinite(self.intercept).all()):
            raise ValueError('invalid frozen weights or free target intercepts')
        self.weights.setflags(write=False);self.intercept.setflags(write=False)

    def predict(self,values):
        if isinstance(values,pd.DataFrame) and
tuple(values.columns.astype(str))!=self.feature_names:
            raise ValueError('raw feature order differs from frozen Q1')
        x=np.asarray(values,float)
        if x.ndim!=2 or x.shape[1]!=len(self.feature_names) or
np.isinf(x).any():
            raise ValueError('predict expects the complete ordered raw feature
matrix')
        stack=np.stack([np.asarray(model.predict(x),float) for model in
self.models],axis=-1)
        if stack.shape[1:]!=self.weights.shape or not np.isfinite(stack).all():
            raise ValueError('base predictions disagree with frozen target/model
dimensions')
        return np.einsum('ntm,tm->nt',stack,self.weights)+self.intercept


def empirical_cdf_grade(
    reference_targets: np.ndarray, predicted_targets: np.ndarray
) -> np.ndarray:
    """Map predictions to [0, 6] using training-only empirical CDFs."""
    reference_targets = np.asarray(reference_targets, dtype=float)
    predicted_targets = np.asarray(predicted_targets, dtype=float)
    if reference_targets.ndim != 2 or predicted_targets.ndim != 2:
        raise ValueError("reference_targets and predicted_targets must be 2-D")
    if reference_targets.shape[1] != predicted_targets.shape[1]:
        raise ValueError("reference and prediction target counts disagree")
    grades = np.empty_like(predicted_targets, dtype=float)
    for target_position in range(reference_targets.shape[1]):
        ordered = np.sort(reference_targets[:, target_position])
        ranks = np.searchsorted(
            ordered, predicted_targets[:, target_position], side="right"
        )
        grades[:, target_position] = 6.0 * ranks / len(ordered)
```

```
    return grades


def verify_archived_prediction(prediction, expected):
    """Match the archive's storage precision, rather than relax an error
gate."""
    prediction=np.asarray(prediction,float);expected=np.asarray(expected)
    if prediction.shape!=expected.shape or not np.isfinite(prediction).all() or
not np.isfinite(expected).all():
        raise ValueError('Finite aligned archived predictions required')
    error=float(np.max(np.abs(prediction-expected)))
    if expected.dtype==np.dtype('float32'):
        if not np.array_equal(prediction.astype(np.float32),expected):
            raise ValueError('Reconstructed prediction differs at archived
float32 precision')
    elif error>1e-9:
        raise ValueError('Reconstructed float64 prediction differs from
archive')
    return error


def migration_model_input(values, source_positions, reference_x):
    values=np.asarray(values,float);raw=np.asarray(reference_x,float)
    positions=np.asarray(source_positions,int)
    if (values.ndim!=2 or raw.ndim!=2 or values.shape[1]!=raw.shape[1]
        or positions.shape!=(len(values),) or np.any(positions<0) or
np.any(positions>=len(raw))
        or np.isinf(values).any() or np.isinf(raw).any()):
        raise ValueError('Aligned raw chemical spectra and source positions
required')
    output=values.copy()
    output[np.isnan(raw[positions])]=np.nan
    return output


def support_audit(x_train, query, *, quantile=.95, outside_fraction=.05):
    train=np.asarray(x_train,float);query=np.asarray(query,float)
    if train.ndim!=2 or query.ndim!=2 or train.shape[1]!=query.shape[1] or
len(train)<2:
        raise ValueError('Training and query matrices must align')
    if np.isinf(train).any() or np.isinf(query).any() or not 0<quantile<1 or
not 0<=outside_fraction<=1:
        raise ValueError('Invalid support geometry')
    median=np.nanmedian(train,axis=0)
    if not np.isfinite(median).all():raise ValueError('A support field has no
observed training values')
```

```
    low,high=np.nanquantile(train,[.25,.75],axis=0)
    scale=np.where(high-low>1e-12,high-low,np.nanstd(train,axis=0))
    scale=np.where(scale>1e-12,scale,1.)
    a=(np.where(np.isnan(train),median,train)-median)/scale
    b=(np.where(np.isnan(query),median,query)-median)/scale

self_distance=cdist(a,a)/np.sqrt(a.shape[1]);np.fill_diagonal(self_distance,np.
inf)
    cutoff=float(np.quantile(self_distance.min(axis=1),quantile))
    distance=cdist(b,a).min(axis=1)/np.sqrt(a.shape[1])
    observed=np.isfinite(query)

out=observed&((query<np.nanmin(train,axis=0))|(query>np.nanmax(train,axis=0)))
    fractions=out.sum(axis=1)/np.maximum(observed.sum(axis=1),1)
    return {'nearest_distance':distance,'distance_cutoff':cutoff,
            'outside_range_fraction':fractions,
            'supported':(distance<=cutoff+1e-12)&(fractions<=outside_fraction)}
```

E.3 浓度转换与缺陷约束（Q3_02_浓度转换与缺陷约束.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q3_02_浓度转换与缺陷约束.py"。

节选自 Q3_02_浓度转换与缺陷约束.py 第 3-102 行。

```
from __future__ import annotations
import numpy as np


def concentrations_from_stored(values, scales):
    values = np.asarray(values, float)
    scales = np.asarray(scales, str)
    if values.ndim != 1 or scales.shape != values.shape or not
np.isfinite(values).all():
        raise ValueError("finite aligned stored values and scales required")
    if not np.isin(scales, ["raw", "log10"]).all():
        raise ValueError("scale must be raw or log10")
    result = values.copy()
    log_mask = scales == "log10"
    result[log_mask] = np.power(10.0, values[log_mask])
    if not np.isfinite(result).all() or (result < 0).any():
        raise ValueError("stored values cannot be represented as nonnegative
concentrations")
    return result


def stored_from_concentrations(values, scales):
    values = np.asarray(values, float)
    scales = np.asarray(scales, str)
```

```
    if values.ndim != 1 or scales.shape != values.shape or not
np.isfinite(values).all() or (values < 0).any():
        raise ValueError("finite aligned nonnegative concentrations and scales
required")
    if not np.isin(scales, ["raw", "log10"]).all():
        raise ValueError("scale must be raw or log10")
    result = values.copy()
    log_mask = scales == "log10"
    if (values[log_mask] <= 0).any():
        raise ValueError("log10 fields require positive concentrations")
    result[log_mask] = np.log10(values[log_mask])
    return result


def direct_limits(source, source_known, threshold, scenario):
    source = np.asarray(source, float)
    known = np.asarray(source_known)
    threshold = np.asarray(threshold, float)
    if scenario not in {"C1", "C2", "C3"}:
        raise ValueError("unknown constraint scenario")
    if source.ndim != 1 or known.shape != source.shape or threshold.shape !=
source.shape or known.dtype.kind != "b":
        raise ValueError("aligned direct-constraint vectors required")
    limits = np.full(source.shape, np.nan)
    valid_source = known & np.isfinite(source)
    if scenario == "C1":
        limits[valid_source] = source[valid_source]
    else:
        valid = valid_source & np.isfinite(threshold)
        if scenario == "C2":
            limits[valid] = np.maximum(source[valid], threshold[valid])
        else:
            limits[valid] = np.minimum(source[valid], threshold[valid])
    return limits


def project_direct_constraints(candidate_stored, source, source_known,
threshold, scales, scenario):
    candidate_stored = np.asarray(candidate_stored, float)
    candidate = concentrations_from_stored(candidate_stored, scales)
    limits = direct_limits(source, source_known, threshold, scenario)
    projected = np.where(np.isfinite(limits), np.minimum(candidate, limits),
candidate)
    return stored_from_concentrations(projected, scales), limits
```

```
def empirical_quantile_anchor(source, source_reference, target_reference):
    source = np.asarray(source, float)
    source_reference = np.asarray(source_reference, float)
    target_reference = np.asarray(target_reference, float)
    if (
        source.ndim != 1
        or source_reference.ndim != 2
        or target_reference.ndim != 2
        or source_reference.shape[1] != source.size
        or target_reference.shape[1] != source.size
        or min(len(source_reference), len(target_reference)) < 2
        or not np.isfinite(source).all()
        or not np.isfinite(source_reference).all()
        or not np.isfinite(target_reference).all()
    ):
        raise ValueError("finite aligned source and style reference matrices
required")
    anchor = np.empty_like(source)
    for column, value in enumerate(source):
        reference = source_reference[:, column]
        percentile = float(np.mean(reference < value) + 0.5 * np.mean(reference
== value))
        anchor[column] = np.quantile(target_reference[:, column], percentile,
method="linear")
    return anchor


def pareto_mask(target_distance, source_change):
    target_distance = np.asarray(target_distance, float)
    source_change = np.asarray(source_change, float)
    if target_distance.ndim != 1 or source_change.shape !=
target_distance.shape:
        raise ValueError("aligned one-dimensional objective arrays required")
    finite = np.isfinite(target_distance) & np.isfinite(source_change)
    result = np.zeros(len(target_distance), dtype=bool)
    for index in np.flatnonzero(finite):
        dominated = finite & (target_distance <= target_distance[index]) &
(source_change <= source_change[index])
        strict = (target_distance < target_distance[index]) | (source_change <
source_change[index])
        if not np.any(dominated & strict):
            result[index] = True
    return result
```

E.4 有限迁移候选与最小改动筛选（Q3_03_受约束迁移求解.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q3_03_受约束迁移求解.py"。

节选自 Q3_03_受约束迁移求解.py 第 3-34 行。

```
import numpy as np
from Q3_02_浓度转换与缺陷约束 import project_direct_constraints


def
constrained_paths(source,anchors,lambdas,feature_indices,source_values,known,th
resholds,scales):

source=np.asarray(source,float);anchors=np.asarray(anchors,float);lambdas=np.as
array(lambdas,float)
    if (source.ndim!=1 or anchors.ndim!=2 or anchors.shape[1]!=len(source)
        or not np.isfinite(source).all() or not np.isfinite(anchors).all()
        or not np.isfinite(lambdas).all() or (lambdas<=0).any() or
(lambdas>1).any()):
        raise ValueError('Finite anchors and strictly positive lambdas in (0,1]
required')

anchor_ids=np.repeat(np.arange(len(anchors)),len(lambdas));path_lambdas=np.tile
(lambdas,len(anchors))
    values=(1-
path_lambdas[:,None])*source+path_lambdas[:,None]*anchors[anchor_ids]
    for row in values:

row[feature_indices],_=project_direct_constraints(row[feature_indices],source_v
alues,known,thresholds,scales,'C1')
    return values,anchor_ids,path_lambdas


def
select_candidates(gain,change,mean_delta,q90_delta,support_fraction,*,minimum_g
ain=.01,guard_margin=.01):
    gain,change,mean_delta,q90_delta,support_fraction=np.broadcast_arrays(
        *[np.asarray(x,float) for x in
(gain,change,mean_delta,q90_delta,support_fraction)])
    if gain.ndim!=1 or not
np.isfinite([*gain,*change,*mean_delta,*q90_delta,*support_fraction]).all():
        raise ValueError('Finite aligned one-dimensional primary diagnostics
required')
    feasible=(gain>=minimum_gain)&(mean_delta<=-guard_margin)&(q90_delta<=-
guard_margin)&(support_fraction>=.9)
    idx=np.flatnonzero(feasible)
```

```
    if not len(idx):return {k:None for k in
['minimum_change','maximum_target_gain','maximum_proxy_margin']},feasible
    # Stable input index breaks exact objective ties; the auditor is not an
argument.
    rules={
        'minimum_change':(idx,-gain[idx],change[idx]),
        'maximum_target_gain':(idx,change[idx],-gain[idx]),
        'maximum_proxy_margin':(idx,change[idx],q90_delta[idx]),
    }
    return {name:int(idx[np.lexsort(keys)[0]]) for name,keys in
rules.items()},feasible
```

E.5 缺陷与代理证据状态（Q3_05_迁移证据状态.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q3_05_迁移证据状态.py"。

节选自 Q3_05_迁移证据状态.py 第 7-113 行。

```
from __future__ import annotations
import numpy as np
import pandas as pd


def audit_concentrations(source, candidate, source_known, scenario: str, *,
                         threshold=None, atol: float = 1e-9, rtol: float = 1e-7)
-> pd.DataFrame:
    if (scenario not in {"C1", "C2", "C3"} or not np.isfinite([atol,
rtol]).all()
            or atol < 0 or rtol < 0):
        raise ValueError("unknown scenario or invalid numerical tolerance")
    source, candidate = np.atleast_1d(np.asarray(source, float)),
np.atleast_1d(np.asarray(candidate, float))
    known = np.atleast_1d(np.asarray(source_known))
    if known.dtype.kind != "b":
        raise ValueError("source_known must contain boolean flags")
    if source.ndim != 1 or source.size == 0 or source.shape != candidate.shape
or source.shape != known.shape:
        raise ValueError("nonempty aligned source/candidate/known vectors
required")
    tau = np.asarray(np.nan if threshold is None else threshold, float)
    tau = np.broadcast_to(tau, source.shape)
    if np.isinf(source).any() or (source[np.isfinite(source)] < 0).any():
        raise ValueError("source must be nonnegative concentration or missing")
    if scenario != "C1" and (np.isinf(tau).any() or (tau[np.isfinite(tau)] <=
0).any()):
        raise ValueError("threshold must be positive or missing")
    limits = source if scenario == "C1" else (
```

```
        np.maximum(source, tau) if scenario == "C2" else np.minimum(source,
tau))
    rows = []
    for before, after, observed, limit, value in zip(source, candidate, known,
limits, tau):
        passed, excess, tolerance = pd.NA, np.nan, np.nan
        if not np.isfinite(after) or after < 0:
            status, passed = "invalid_candidate", False
        elif not observed or not np.isfinite(before):
            status = "unknown_source"
        elif scenario != "C1" and not np.isfinite(value):
            status = "unknown_threshold"
        else:
            tolerance = atol + rtol * abs(before)
            excess = float(max(after - limit, 0.))
            passed = bool(after <= limit + tolerance)
            status = "pass" if passed else "violation"
        rows.append({"scenario": scenario, "source": before, "candidate": after,
                     "limit": limit, "threshold": value, "status": status,
                     "violation_magnitude": excess, "numerical_tolerance":
tolerance,
                     "passed": passed})
    frame = pd.DataFrame(rows)
    frame["passed"] = pd.array(frame["passed"], dtype="boolean")
    return frame


def project_upper_bounds(candidate, upper) -> np.ndarray:
    candidate, upper = np.broadcast_arrays(np.asarray(candidate, float),
np.asarray(upper, float))
    if (not np.isfinite(candidate).all() or (candidate < 0).any()
            or np.isinf(upper).any() or (upper[np.isfinite(upper)] < 0).any()):
        raise ValueError("projection requires finite nonnegative candidates and
valid bounds")
    return np.where(np.isfinite(upper), np.minimum(candidate, upper),
candidate)


def summarize_migration(audit: pd.DataFrame, *, target_gain: bool,
                        proxy_nonworsening: bool | None) -> dict:
    if (not isinstance(target_gain, (bool, np.bool_)) or
            (proxy_nonworsening is not None and not
isinstance(proxy_nonworsening, (bool, np.bool_)))):
        raise ValueError("migration/proxy decisions must be explicit boolean
flags or unknown")
    if audit.empty or not {"status", "passed"}.issubset(audit):
```

```
        raise ValueError("nonempty direct-constraint audit required")
    passed = audit.passed.astype("boolean")
    n_passed, n_failed = int(passed.fillna(False).sum()), int((passed ==
False).fillna(False).sum())
    n_unknown = int(passed.isna().sum())
    if n_failed:
        status = "rejected_direct_constraint"
    elif not target_gain:
        status = "no_target_gain"
    elif n_unknown:
        status = "needs_direct_verification"
    elif proxy_nonworsening is None:
        status = "needs_proxy_verification"
    elif not proxy_nonworsening:
        status = "rejected_proxy"
    else:
        status = "eligible_statistical_candidate"
    return {"n_checks": len(audit), "n_passed": n_passed, "n_failed": n_failed,
            "n_unknown": n_unknown,
            "decidable_pass_rate": n_passed / (n_passed + n_failed) if n_passed
+ n_failed else None,
            "status": status, "eligible": status ==
"eligible_statistical_candidate",
            "physical_safety_certified": False}


def proxy_screen(primary_deltas, audit_deltas, primary_reliable,
audit_reliable, *, tolerance=1e-8):
    p=np.asarray(primary_deltas,float);a=np.asarray(audit_deltas,float)
    if p.shape!=(10,) or a.shape!=(10,) or not
np.isfinite([*p,*a,tolerance]).all() or tolerance<0:
        raise ValueError('Ten finite paired-repeat changes and a numerical
tolerance required')

pm,am=float(p.mean()),float(a.mean());pq,aq=float(np.quantile(p,.9)),float(np.q
uantile(a,.9))
    if not primary_reliable or not audit_reliable:status='proxy_unreliable'
    elif (pm>tolerance)!=(am>tolerance):status='proxy_disagreement'
    elif pm>tolerance or am>tolerance:status='predicted_worsening'
    elif pq>tolerance or aq>tolerance:status='proxy_variation_uncertain'
    else:status='predicted_nonworsening'
    return
{'proxy_status':status,'primary_mean_delta':pm,'audit_mean_delta':am,
            'primary_repeat_q90_delta':pq,'audit_repeat_q90_delta':aq,
            'physical_safety_certified':False}
```

```
def candidate_status(*, available, n_failed, n_unknown, target_gain,
proxy_status, support_fraction):
    if not available:return 'rejected_no_structural_candidate'
    if n_failed>0:return 'rejected_direct_constraint'
    if not target_gain:return 'no_target_gain'
    if n_unknown>0:return 'needs_direct_verification'
    if support_fraction<.9:return 'outside_predictor_support'
    if proxy_status!='predicted_nonworsening':return proxy_status
    return 'eligible_statistical_candidate'
```

E.6 分布与相关结构诊断（Q3_06_分布一致性检验.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q3_06_分布一致性检验.py"。

节选自 Q3_06_分布一致性检验.py 第 3-59 行。

```
from __future__ import annotations
import numpy as np
from scipy.spatial.distance import cdist
from scipy.stats import rankdata, wasserstein_distance


def distribution_diagnostics(real, generated, training, scale, *, knn_k: int =
1,
                             duplicate_tolerance: float = 1e-6) -> dict:
    arrays = [np.asarray(a, float) for a in [real, generated, training]]
    if any(a.ndim != 2 or len(a) < 2 or not np.isfinite(a).all() for a in
arrays):
        raise ValueError("finite matrices with at least two rows required")
    real, generated, training = arrays
    p = real.shape[1]
    if not p or generated.shape[1] != p or training.shape[1] != p or
len(real) != len(generated):
        raise ValueError("matched real/generated sample counts and field
dimensions required")
    scale = np.asarray(scale, float)
    if scale.shape != (p,) or not np.isfinite(scale).all() or (scale <=
0).any():
        raise ValueError("positive finite training-derived scale required")
    if (not isinstance(knn_k, (int, np.integer)) or not 1 <= knn_k < len(real)
            or not np.isfinite(duplicate_tolerance) or duplicate_tolerance < 0):
        raise ValueError("invalid neighbor count or duplicate tolerance")
    with np.errstate(over="ignore", invalid="ignore"):
        r, g, train = real / scale, generated / scale, training / scale
    if not all(np.isfinite(a).all() for a in [r, g, train]):
        raise ValueError("training-scale normalization is not numerically
representable")
```

```
    rms_factor = np.sqrt(p)
    rg, rr, gg = [a / rms_factor for a in [cdist(r, g), cdist(r, r), cdist(g,
g)]]
    if not all(np.isfinite(a).all() for a in [rg, rr, gg]):
        raise ValueError("pairwise distances are not numerically representable")
    w1 = [float(wasserstein_distance(r[:, j], g[:, j])) for j in range(p)]
    nonconstant = (np.ptp(r, axis=0) > 0) & (np.ptp(g, axis=0) > 0)
    n_cols = int(nonconstant.sum())
    n_pairs = n_cols * (n_cols - 1) // 2
    correlation_error = np.nan
    if n_pairs:
        cr = np.corrcoef(rankdata(r[:, nonconstant], axis=0), rowvar=False)
        cg = np.corrcoef(rankdata(g[:, nonconstant], axis=0), rowvar=False)
        upper = np.triu_indices(n_cols, k=1)
        correlation_error = float(np.sqrt(np.mean((cr[upper] - cg[upper])**2)))
    rr_no_self = rr.copy()
    np.fill_diagonal(rr_no_self, np.inf)
    radii = np.partition(rr_no_self, knn_k - 1, axis=1)[:, knn_k - 1]
    nearest_train = cdist(g, train).min(axis=1) / rms_factor
    return {
        "n_real": len(r), "n_generated": len(g), "n_training": len(train),
        "n_features": p, "knn_k": int(knn_k),
        "mean_w1": float(np.mean(w1)), "max_w1": float(np.max(w1)),
"w1_by_feature": w1,
        "energy_v": float(max(2 * rg.mean() - rr.mean() - gg.mean(), 0.)),
        "spearman_pair_rmse": correlation_error, "n_spearman_pairs": n_pairs,
        "n_constant_real": int((np.ptp(r, axis=0) == 0).sum()),
        "n_constant_generated": int((np.ptp(g, axis=0) == 0).sum()),
        "coverage": float(np.mean(rg.min(axis=1) <= radii)),
        "near_duplicate_fraction": float(np.mean(nearest_train <=
duplicate_tolerance)),
        "median_nearest_training_rms_distance": float(np.median(nearest_train)),
        "duplicate_tolerance": float(duplicate_tolerance),
        "scope": "descriptive_distribution_diagnostics_no_hypothesis_test",
    }
```

E.7 指定感官目标的组合预测（Q3_04_感官画像推理.py）

软件：Python 3.10.19、NumPy 2.2.6、SciPy 1.15.3、pandas 2.3.3、scikit-learn 1.7.2。语法检查命令：python -m py_compile "A2601605 源程序/Q3_04_感官画像推理.py"。

节选自 Q3_04_感官画像推理.py 第 107-124 行。

```
def predict_single_target(model,values,target):
    """Same frozen NNLS computation, retaining just the requested output
column."""
    x=np.asarray(values,float)
    if x.ndim!=2 or x.shape[1]!=len(model.feature_names) or np.isinf(x).any():
        raise ValueError('Full raw feature matrix required')
```

```
    out=[]
    # Multi-output tree shares splits, so use its original prediction path.
    out.append(model.models[0].predict(x)[:,target])
    hgb=model.models[1]
    out.append(hgb.models[target].predict(hgb.imputer.transform(x)))
    ridge=model.models[2];transformed=ridge[:-1].transform(x)
    out.append(transformed @ ridge[-1].coef_[target]+ridge[-
1].intercept_[target])
    pls=model.models[3];transformed=pls[:-1].transform(x);estimator=pls[-1]
    # PLS coefficients multiply its centered X input (sklearn predict
convention).
    out.append((transformed-estimator._x_mean) @
estimator.coef_[target]+estimator.intercept_[target])
    values=np.column_stack(out) @ model.weights[target]+model.intercept[target]
    if not np.isfinite(values).all():raise ValueError('Non-finite frozen
single-target prediction')
    return values
```